Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
National Engineering of School in Tunis

DEPARTMENT OF ELECTRICAL ENGINEERING

Graduation Project

Presented by
Mohamed Hedi BOUSSAMA

to obtain the
National Diploma in Electrical Engineering

Embedded diagnostic and support solution in vehicles
BDiag

Elaborated at
ACTIA Engineering Services

Presented on 11/09/2019

In front of the Juries:

President	**:**	**Mr.**	**Riadh BOURGHIBA**
Report Reviewer	**:**	**Mr.**	**JOSEPH HAGGEGE**
Host Company Supervisor	**:**	**Mr.**	**Bilel BENHALIMA**
ENIT Supervisor	**:**	**Mr.**	**Lotfi CHARAABI**

Academic year: 2018/2019

CERTIFICAT D'ANALYSE

Compte : **Mohamed-El-Hedi BOUSSAMA**
Identifiant : **tl2ora5d**
Titre : **Rapport.pdf**
Dossier : **Dossier par défaut**
Commentaires : *Non renseigné*
Chargé le :09/09/2019 09:11

Similitudes document :


9%

Similitudes partie 1 :


9%

INFORMATIONS DÉTAILLÉES

Titre : rapport.pdf
Description : [Partie 1]
Analysé le : 09/09/2019 09:42
Identifiant : dyzjmfol
Chargé le : 09/09/2019 09:11
Type de chargement : Remise manuelle des travaux
Nom du fichier : rapport.pdf
Type de fichier : pdf
Nombre de mots : 9483
Nombre de caractères : 48197

TOP DES SOURCES PROBABLES *- PARMI 4 SOURCES PROBABLES*

1. raks.ee/.../pdf/exploringraspberrypi.pdf — 5%
2. mechanicbase.com/.../reviews/best-obd2-scanner — <1%
3. www.generalautorepairsd.com/.../what-is-a-car-diag...is-it-so-important — <1%

SIMILITUDES TROUVÉES DANS CE DOCUMENT/CETTE PARTIE

Similitudes à l'identique : **5 %**
Similitudes supposées : **4 %**
Similitudes accidentelles : **<1 %**

Sources très probables - 4

Sources accidentelles- 3 Sources

CERTIFICAT D'ANALYSE

Compte : **Mohamed-El-Hedi BOUSSAMA**
Identifiant : **tl2ora5d**
Titre : **Rapport.pdf**
Dossier : **Dossier par défaut**
Commentaires : *Non renseigné*
Chargé le :09/09/2019 09:10

Similitudes document :


9%

Similitudes partie 2 :


10%

INFORMATIONS DÉTAILLÉES

Titre : rapport.pdf
Description : [Partie 2]
Analysé le : 09/09/2019 09:42
Identifiant : wuvtyjds
Chargé le : 09/09/2019 09:10
Type de chargement : Remise manuelle des travaux
Nom du fichier : rapport.pdf
Type de fichier : pdf
Nombre de mots : 7400
Nombre de caractères : 46436

TOP DES SOURCES PROBABLES - *PARMI 13 SOURCES PROBABLES*

1. **iot-traffic-light-control.readthedocs.io**/.../latest/software.html — 5%
2. **www.yoctoproject.org**/.../yocto-project-qs/yocto-project-qs.html — 5%
3. **www.phytec.de**/.../Manuals/L-813e_2.pdf — <1%
4. **linoxide.com**/.../how-tos/d-bus-ipc-mechanism-linux — <1%
5. **www.phytec.de**/.../phyBOARD-Mira_i.MX6/L-806e_1.pdf — <1%
6. **www.yoctoproject.org**/.../ — <1%

SIMILITUDES TROUVÉES DANS CE DOCUMENT/CETTE PARTIE

Similitudes à l'identique : **7 %**

Acknowledgement

I would like to take advantage of this report to express my heartfelt thanks to anyone who contributed to the development of this work.

I would like to think **ACTIA ES** for the opportunity they offered to me to integrate in professional world.

In particular, I would like to thank my supervisor Mr. **Bilel BENHALIMA**, Manager of ACTIA Nordic, who has managed to trust me with missions that challenged my knowledge.

I would also like to thank **Mohamedtaher MASMOUDI** and **Rim MSALMI**, embedded software engineer, for their valuable advice's and explanations throughout this project.

Besides, I would like to thanks my supervisors, **Mr.Lotfi CHAARABI** and **Mr. Samir SAKRANI** , for the continuous support and advice and valuable comments.

Table of Contents

List of Figures iv

List of Tables vi

List of Acronyms vii

Abstract viii

General Introduction 1

I Project Context 3

1 Presentation of hosting company - ACTIA ES 4

2 Project Framework 6

2.1 Project context : Diagnostic 6

2.2 Existing solutions 7

2.3 Problematic 11

3 Internship diary and Working methodology 11

3.1 SCRUM methodology 11

3.2 Project time-line 13

4 General Architecture 14

4.1 Presentation of BDiag solution 14

4.2 BDiag Architecture 14

5 Requirement Analysis and Specification 15

5.1 Specification of functional requirements 15

5.2 Specification of Non-functional requirements 16

6 System specification and Requirement modelling 17

II Yocto Project 22

1 Embedded Linux 22

1.1 Advantages of Embedded Linux 24

1.2 The four elements of Embedded Linux 24

1.3 Toolchains 25

1.3.1 Types of toolchain 26

1.3.2 Finding a toolchain 27

2 Yocto project 27
2.1 Core Components 28
2.2 YP Development Environment 29
3 Specific image for the phyBOARD-Mira 30
3.1 Target device : phyBOARD-Mira i.MX6 30
3.2 Create embedded Linux image for the phyBOARD Mira 34

III Implementation of BDiag Solution **38**
1 BDiag user interface : "BDiagGUI" 38
1.1 Connect with the Mira Serial / SSH 39
1.2 Qt : Graphical development environment 40
1.3 Cross-Compiling Qt for phyBOARD-Mira 40
1.4 BDiagGUI interface 43
1.4.1 QML Syntax 43
1.4.2 BDiag GUI App 43
2 Using Git for BDiag solution 47
3 The engine of BDiag solution : "BDiagApp" 49
3.1 Inter-process communication with D-Bus 49
3.1.1 D-Bus advantages 50
3.1.2 D-Bus on BDiag solution 50
3.2 Controller Area Network for ECU communication 52
3.3 CAN Bus on the phyBOARD-Mira 53

General Conclusion **57**

References **59**

Annex A : BDiag Mind Mapping **61**

Annex B : phyCORE-i.MX6 **62**

Annex C : phyBOARD-Mira i.MX6 **63**

Annex D : Git shell commands **64**

List of Figures

I.1 ACTIA ES Head Office 4
I.2 ACTIA ES Organization Chart 5
I.3 Development of electronic networking W211 (1995) and (2002) 6
I.4 BlueDriver 7
I.5 ANCEL AD410 8
I.6 Autel MaxiCom MK808 8
I.7 Multi-Diag equipment 9
I.8 Multi-Diag software and services[2] 10
I.9 Remote diagnostic architecture[3] 10
I.10 Scrum methodology cycle 12
I.11 Scrum methodology cycle 13
I.12 BDiag General Architecture 14
I.13 Global Use Case of the BDiag solution 17
I.14 Diagnostic Use Case diagram 18
I.15 Connections Use Case diagram 19
I.16 Sequence Diagram : Opening 19
I.17 Sub Sequence diagram : Connect 20
I.18 Use Case Diagram : Configurations 21

II.1 Embedded systems 23
II.2 Toolchain components 25
II.3 Toolchain types 26
II.4 Yocto project logo 27
II.5 Yocto project origin 28
II.6 YP Architecture Workflow 29
II.7 phyBOARD Mira i.MX 6 with phyCORE i.MX 6 31
II.8 phyBOARD-Mira Connector Interfaces 32
II.9 phyBOARD Mira Expansions 33
II.10 YP working directory 34
II.11 Build configuration "local.conf" 35
II.12 YP environment setup 35
II.13 Bitbake CPU/RAM usage 36

III.1 Serial comm 39
III.2 Qt logo 40
III.3 Qt SSH configuration 41
III.4 Qt development Kits 42
III.5 GUI main window 44
III.6 Diagnostic window 44
III.7 Full Diagnostic report window 45
III.8 Manual Diagnostic window 45
III.9 Configuration window 46
III.10BDiag Git project view 47
III.11BDiagApp repositories Git view 48
III.12DBus 50
III.13d-feet view 51
III.14CAN Bus 52
III.15CAN wave 53
III.16Mira CAN Bus output 54
III.17Mira and telematic ECU frames exchange 54
III.18BDiag on Multiples platform 55

A.1 BDiag Mind Mapping 61

D.1 Git shell commands 64

List of Tables

List of Acronyms

API : Application Programming Interface
App : Application
BSD : Berkeley Software Distribution
BSP : Board support package
CAN : Controller Area Network
CC : Cross Compiler
DTC : Diagnostic Trouble Code
ECU : Electronic Control Unit
GPS : Global Positioning System
GUI : Graphical User Interface
IDE : Integrated Development Environment
IPC : Inter-Process Communication
RAM : Random-access memory
SOS : Signal code uses in emergencies
SSD : Solid-State Drive
SSH : Secure Shell
POSIX : Portable Operating System Interface
OTA : Over-the-Air
UI : User Interface
UML : Unified Modeling Language
UX : User experience
VCI : Vehicle Communication Interfaces
YP : Yocto Project

Abstract

Nowadays, vehicles are becoming more and more complex and present a variety of features, that help us not only control the vehicle but enjoy the driving experience. For that, hundred of on-board computers called also Electronic Control Units (ECU) needed to manage all the feature. In the same context, diagnostics is becoming a real challenge especially when it comes to locating and detecting the source of problems with the presence of hundreds of sensors, wires and calculators.

The present project aims to create a new approach of in vehicle diagnostic that take advantages of new technologies.

Key words: **Diagnostic, Embedded system, Linux, Telematic ECU, D-Bus.**

Resume

Aujourd'hui, les vehicules sont de plus en plus complexes et presentent une variete de caracteristiques, qui nous aident non seulement a controler le vehicule, mais profiter de l'experience de conduite. Pour cela, une centaine d'ordinateurs de bord appeles aussi Electronic Control Units (ECU) necessaires pour gerer toute la fonctionnalite. Dans le meme contexte, le diagnostic devient un veritable defi, surtout lorsqu'il s'agit de localiser et de detecter la source des problÃ¨mes lies a la presence de centaines de capteurs, de fils et de calculatrices. Le present projet vise a creer une nouvelle approche du diagnostic dans les vehicules qui tirent parti des nouvelles technologies.

Key words : **Diagnostic, Embedded system, Linux, Telematic ECU, D-Bus.**

General Introduction

The introduction of embedded systems in vehicles is considered one of the undeniable technological achievements of the automotive industry. Indeed, these embedded systems have been able to meet the increasing requirements of the customer by comfort, safety, efficiency, driving experience and to overcome the increasingly strict constraints in order to reduce consumption, pollution and making the roads a better place at all levels.

In the automotive industry where novelty and originality are essential arguments for commercialization, embedded electronics represents the major part of automotive innovation. The importance of electronics is such that, nowadays, a car leaving the factory has several hundred on-board computers called also Electronic Control Units (ECU). These computers, distributed throughout the vehicle volume, host many functions and perform more than a million lines of software code. As an illustration, in 1985, a car embedded an injection computer with 10 functions. Today, a Renault Onboard space holds 25 electronic computers performing 90 functions.

On the other hand, the complexity of electronic architectures is growing more and more according to the empirical law of more. So the automobile industry suffers from problems of complexity are due to the diversity of networks and computers that are among the bodies with shorter lifetimes in a vehicle.

With that being said, many challenges emerge in automotive engineering. One on those is the diagnostic and maintenance of the vehicles and their components. This process will detect the anomalies in a car in terms of technical problems, perhaps identify the sources and causes to overcome those with the proper maintenance solutions.

With the growth of the market of connected devices, many companies aim to develop such end-to-end solutions to gain such promising domain and provide compatible and portable diagnostic services of vehicles on the road to the goal which is conceiving a fully connected smart car, this ultimate future dream of visionaries companies in the field of automotive industry.

In our project, we aim to put on evidence a new approach of on-board diagnostic system, that will take advantage of the development in automotive sector. Also, with this project, we want to enhance the drive experience by creating a new connected service that guaranty more the safety of both users and vehicles.

This present project is just a prototype to validate the solution design and technologies to be used. That ?s why it is going to be a common solution. Moreover, this prototype will be upgradable and customizable to fit into any vehicle, in the future, and fulfil its needs.

In this context, this report presents the different aspects of this project as well as the development steps of the proposed solution in the aim of conceiving a prototype of a marketable solution. This document will be divided into three chapters.

The first chapter is devoted to expose the general framework of the project, where we talk about the problematic that we want to solve with this project. This will be succeeded by exposing the proposed solution as well as its basic architecture. we will give more detail about the solution through a deep requirement analysis and fix of the functional and non-functional specification of the solution.

In a second chapter, we will talk about Embedded Linux, its advantages and how its becoming a standard in every day life embedded systems. Besides, we will discuses the Yocto Project concept and will shows how this build system can be an articular part in this project. Then we will use Yocto build system to create a customized Linux image for our embedded board.

At the third and the last chapter, we will pass through the different steps of the implementation of our solution. where we will highlight some of the most important notions in embedded system world, the inter-process communication and CAN protocol.

Chapitre I

Project Context

Plan

1 Presentation of hosting company - ACTIA ES 4

2 Project Framework 6

2.1 Project context : Diagnostic 6

2.2 Existing solutions 7

2.3 Problematic 11

3 Internship diary and Working methodology 11

3.1 SCRUM methodology 11

3.2 Project time-line 13

4 General Architecture 14

4.1 Presentation of BDiag solution 14

4.2 BDiag Architecture 14

5 Requirement Analysis and Specification 15

5.1 Specification of functional requirements 15

5.2 Specification of Non-functional requirements 16

6 System specification and Requirement modelling 17

Introduction

This first chapter will be a general introduction of the global context of this final internship. Where we will present in its first part the hosting company, the project Framework, the working strategy including both (the plan and the methodology).
In its second part, will be an overview of the solution by identifying the requirement for the development of "*The embedded diagnostic and support solution in vehicles (**BDiag**)*", including the functional and non-functional requirements of the application, and by modelling the requirements based on UML diagrams.

1 Presentation of hosting company - ACTIA ES

ACTIA Engineering Services (ex ARDIA] is an engineering consulting company, that was created in 2005. ARDIA, the acronym of "ACTIA Recherche et Développement en Informatique Appliquée", is a subsidiary of ACTIA Group, a well-known French group.
With more than 20 subsidiaries in 15 countries, ACTIA Group is specialized in electronic equipment with a high-leveled value targeting the market of automotive and telecommunications.



Figure I.1 – ACTIA ES Head Office

ACTIA ES has extensive experience thanks to the achievement of many projects in an international context. Actually, since its creation, ACTIA ES supports its customers in the development and integration of embedded electronics and related software. ARDIA has developed real expertise in :

— The development of WEB applications and mobile solutions
— The mechanical and electronic studies
— Testing and validation of complex systems
— The making of some bus dashboard

Nowadays, ACTIA ES rents about 400 employees, and it has implemented an effective quality system, which is in continues improvement, as well as methodologies and project management expertise that arise from the best practices and the highest standards of the software industry.

I.1 Presentation of hosting company - ACTIA ES

ACTIA ES is ISO 9001 certified in 2008 for all its activities such as design, development, test, and validation of diagnostics' tools, mechanical systems, production tooling for automotive applications, industrial support, and qualification test. It has, also, a thermomechanical qualification laboratory that was accredited according to EN ISO 17025 :2005.
The figure I.2 below presents the different departments in ACTIA ES. The present project was established in "Actia Nordic" team. This team counting around thirty engineer, that working mostly on Telematic ECU.

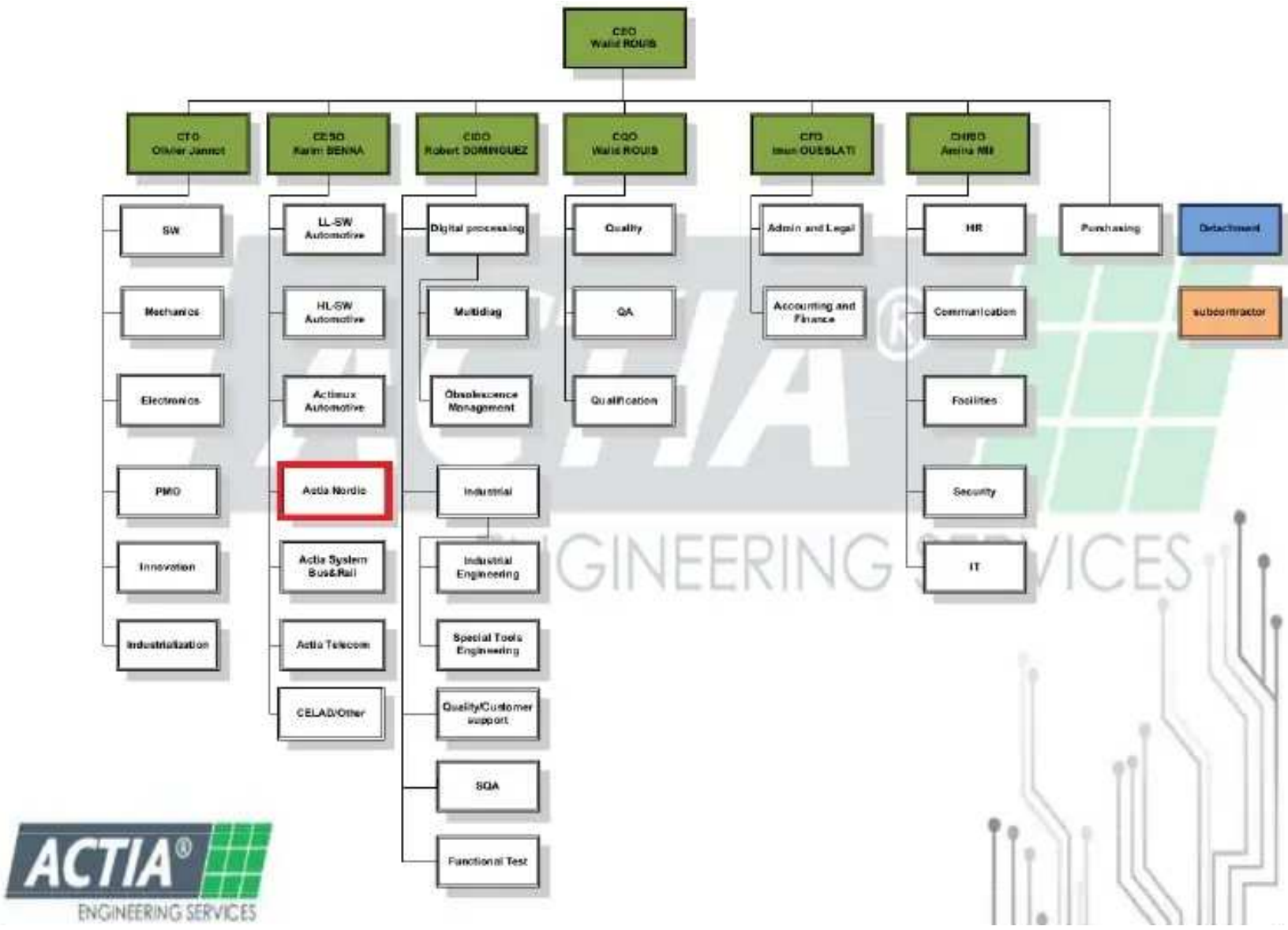


Figure I.2 – ACTIA ES Organization Chart

2 Project Framework

In this part, we will expose the general problematic that we try to solve through this project. But also, we will enumerate the existing solutions presented on the market with an analysis of the advantages and weaknesses of each one.

2.1 Project context : Diagnostic

Nowadays, vehicles are becoming more and more complex and present a variety of features, that help us not only control the vehicle but enjoy the driving experience.
In fact, at the present time a vehicle offers more than a simple ride and the basic definition of a car that have four wheels, engine, transmission and a steering wheel to control it, is no longer valid. Because cars offer more than that, and what's considered as a luxury in the past, is becoming a necessity at the present. As an example, ABS (Anti-lock Braking System) which was introduced in the 1950s as a safety invention is becoming a standard in all vehicles even on bikes.In addition, there are more advanced options like the Advanced driver-assistance systems (ADAS) which is an electronic systems that help a vehicle driver while driving, the self-driving ability of future cars or Autopilot...etc.
So, it's clear that the three bus nodes in commercial vehicles in 1995 (see figure I.3) is no more accurate, today in similarly equipped vehicles more than hundreds calculator can be used to handle all the features within the vehicle.



Figure I.3 – Development of electronic networking W211 (1995) and (2002)

In the same context, diagnostics is becoming a real challenge especially when it comes to locating and detecting the source of problems with the presence of hundreds of sensors, wires and calculators.

A vehicle diagnostic test can reveal a number of issues related with vehicle's engine and transmission, oil and gas tanks, exhaust system, and other systems.
Today's vehicles are designed with computer processors, microchips, and sensors that can be linked to a computer which scans the operating systems to pinpoint exactly where a problem may exist. They have the capability to produce a large variety of fault codes which get stored in the vehicle's memory system. Several cars have some units which control engine management, brakes, suspension and even windscreen wipers.
With diagnostic equipment, it is possible to identify the correct cause of the fault or problem and rectify it before the problem escalates. Otherwise, without diagnostic tools it would be difficult and time consuming to detect problems and identify their origins.

Diagnostic tools save time, eliminate guesswork while troubleshooting and helps ensure the vehicle's problem is properly addressed. Diagnostic tools can also identify parts and systems that are failing before they become a more expensive and catastrophic problem.

2.2 Existing solutions

• **BlueDriver :** It is considered as one of the most cheaper scanners in the market. It can only read the generic trouble code memory from the engine control unit, and if you are going to read the SRS, ABS and other control units, you have to check for a more expensive scanner. The BlueDriver figure I.4 can scan the different control units for the most known car models.



Figure I.4 – BlueDriver

• **ANCEL AD410 :** A generic OBD2 Reader, It's a quite simple DTC tool that will do the job for you if you are only want to read the generic engine DTC codes. It can also read Live data (Parameters of the engine). It can also read some manufacturer trouble codes. It has an inbuilt system for information about trouble codes you get from your engine.



Figure I.5 – ANCEL AD410

• **Autel MaxiCom MK808 :** For Professionals, a real high performing diagnostic tool for automotive car workshops. It's probably over the budget of the most home users, and it may be a better choice for a small workshop.
The Autel MaxiCom MK808 figure I.6 can read the most different control units in most of the car brands on the market, and it have the required coding and programming functions like service light reset, electric parking brake reset, key programming and much more.



Figure I.6 – Autel MaxiCom MK808

• **ACTIA Multi-Diag :** Multi-Diag solutions concentrate all of ACTIA's expertise which is dedicated to professionals in the automotive aftermarket. In addition, the group offers an extensive range of garage equipment and applications for connected workshops and vehicles [2]. The ACTIA diagnostic solution composed by three separate components :
- VCI : Vehicle Communication Interface figure I.7a, it's the interface between the vehicle and the diagnostic tablet or PC. It can communicate with Bluetooth or USB without the need of additional OS.
- Mobile 3 : (figure I.7b) is the "all-terrain" tool that supports users throughout the day. Mainly it's a 11" tablet powered by Intel Atom processor, 2 Gb RAM and 64 Gb SSD storage, that communicate with the VCI to offer a complete diagnostic.
- XG Master : (figure I.7c) is the high-end diagnostic tool from ACTIA. It's a real PC, equipped with an Intel Core I7 processor, rechargeable hot-plug batteries and 13.3" touch screen.
XG Master also offers users the best performance and allows to make the most of Multi Diag 360 all within a windows environment that is open to other applications.



(a) VCI **(b)** mobile3 360 **(c)** XG Master 360

Figure I.7 – Multi-Diag equipment

Besides this equipment, the Multi-Diag solution is based on Multi-Diag 360 software, which can be delivered in four different pack : Basis, SOS, Essential and Expert, as it shown in the next figure I.8.
In addition ACTIA offers many useful services. Like the "Actia Connect" to send a diagnostic report to the repair center so an expert can analyze well the vehicle status. Also "Hotline" service can be a strong option, in which the users can interact with an expert after a diagnostic session to decide what's next step they should take to repair their vehicles.

SOFTWARE & SERVICES

	Ref.	Multi-Diag 360°	Practical cases	Guided Method	Technical Doc.	Parts Info	Actia Connect	Hotline	Vehicle Repair Centre Hotline
Basic	AC600278	•				•	•	•	option
SDS	AC600279	•	•			•	•	•	option
Essential	AC600280	•	•	•		•	•	•	option
Expert	AC600281	•	•	•	•	•	•	•	option

EQUIPMENT & SERVICES

	Ref.	Vehicle Communication Interface	Tablet	Actia Connect	Hotline	Standard exchange in 48 hours	Warranty (years)
Access	AC965831	•		•	•	•	1
Mobile 3	AC967600	•	•	•	•	•	3
XG Master	AC966770	•	•	•	•	•	3

Figure I.8 – Multi-Diag software and services[2]

• **Remote diagnostics :** Remote diagnosis system for vehicle becoming a comparative new product in consumer market. The idea, described by figure I.9, is to make vehicle communicate through wireless links (such as 3G) send message from the power system and CAN bus of remote diagnosis to remote diagnostic center.

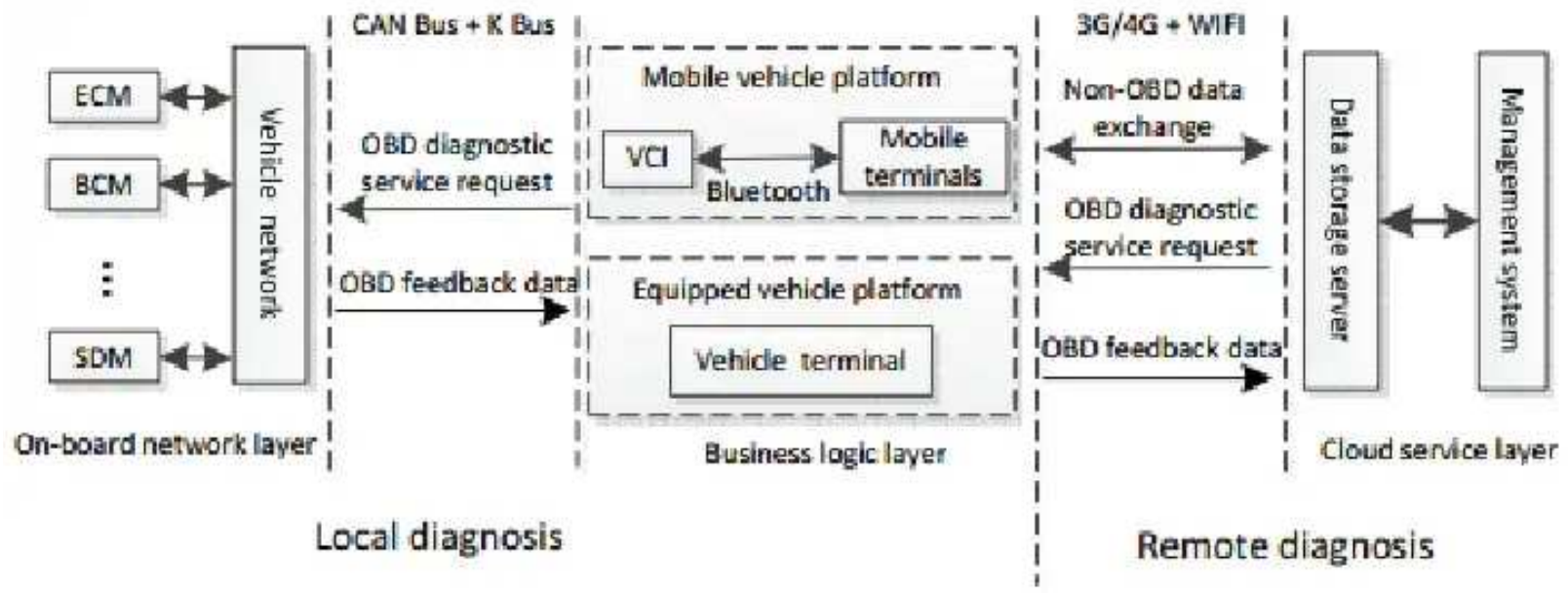


Figure I.9 – Remote diagnostic architecture[3]

After the remote diagnosis center received the message from the vehicle, remote database and other resources integrate, analyse, and determine the information on the vehicle using the expert

system in central server, and then operation should be given contrapose the failure. Treatment results will be returned to terminal through the wireless communication and displayed to the driver, to assist the driver to achieve purpose of processing in time and avoiding accidents. We must note here, that there is many related research focused at this subject [4] [5] and [6], as well as, many companies opted this concept of remote diagnostic, by creating a specific service for their product like VOLVO Management Platform [7] and Scania Assistance [8].

2.3 Problematic

Many important systems in new generation vehicles are computer controlled. Without high-tech diagnostic tools, it's near impossible to tell if the vehicle's problem is caused by a failing system or a bad computer. Within seconds, the diagnostic tools read, analyses the faults code and send out a notification signal to the main computer, giving a clear idea where the problem occurs. Without these tools, accurately diagnosing malfunctions would take much more time and be very costly.

However, a common misconception about car diagnostic tests is that technicians can use code-reading tools to determine the exact problem that triggered the check engine light. In reality, the code tells technicians which engine or component parameters are out of range, but it does not detail the cause of the problems.

As we enumerated in the existent solution section, both cheaper and professional automotive diagnostic scanners are available. But they are all considered to be an external tool and sometimes they need user knowledge to interpret the DTC codes, especially for cheaper products. So, the main purpose of this project is to create a new diagnostic solution that is fully integrated in the car itself and that gives enhanced diagnostic message comprehensive by ordinary vehicles drivers.

3 Internship diary and Working methodology

In order to ensure effective progress monitoring of the project, a good planning is necessary as well as a strong methodology. In this project we choose to work following Scrum methodology which is one of many Agile methods.

3.1 SCRUM methodology

Agile methods have emerged since the 1990s as a result of the failure of several projects using classical methods and have become alternatives to old one such as the cascading method.

These methods ensure a good estimation of costs and deadlines. They use an iterative principle that consists in dividing the project into several minis-projects accompanied by customer listening and tests throughout the development cycle.

SCRUM which is part of the agile methodologies and has been adopted by our team - BDiag team - for the management of the project.
SCRUM is an agile framework in which complex problems can be addressed and it provides a product with maximum effectiveness and creativity. SCRUM divides a project (which described with features), into many sprints, each one can be done between two and four weeks.

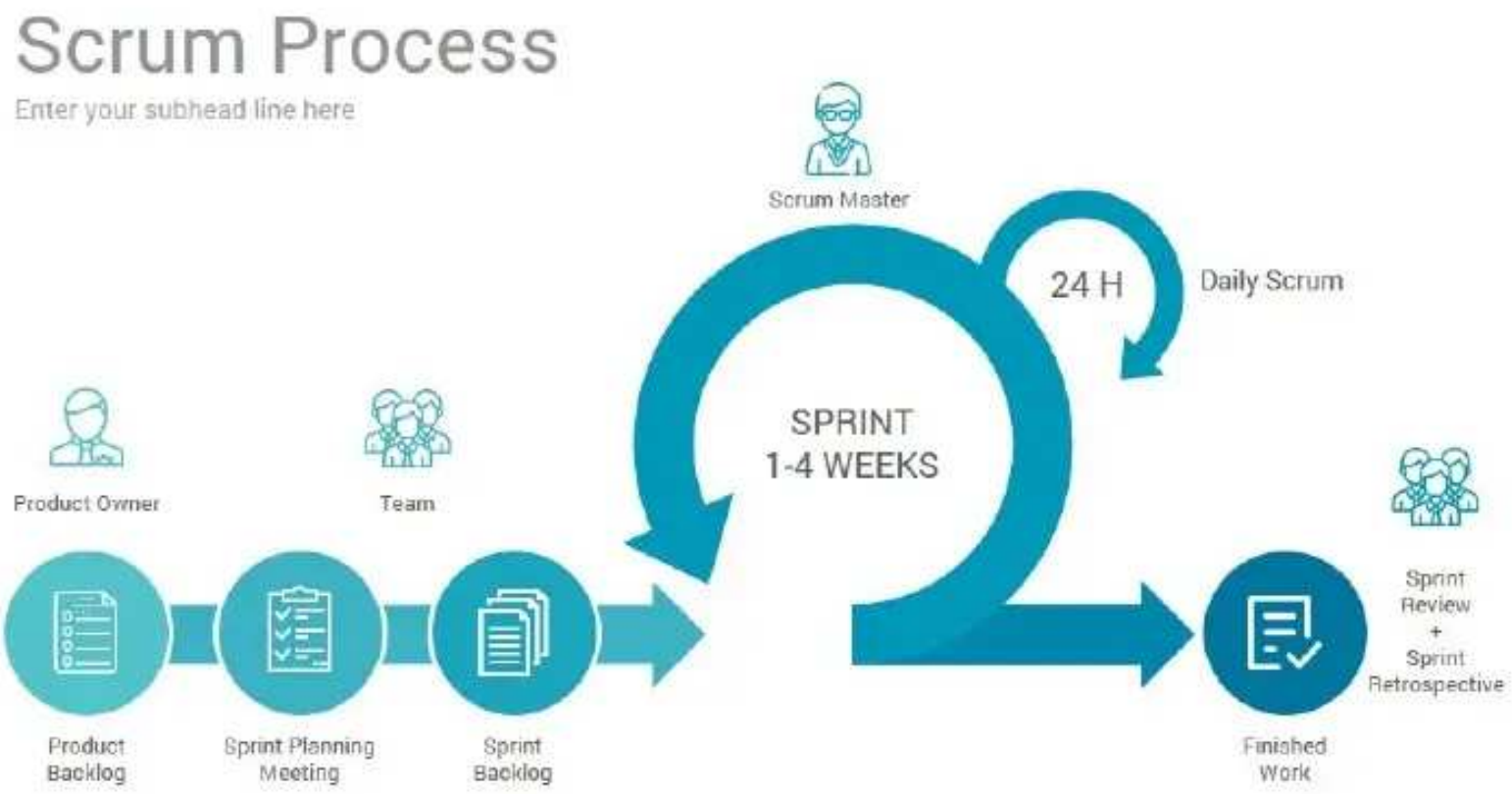


Figure I.10 – Scrum methodology cycle

To achieve SCRUM, three values must be ensured : *Transparency, Inspection and Adaptation.* SCRUM offers three levels of operational planning as illustrated in the Figure below I.11 Sprint, Release and Quotidian.

— Sprint : Scrum is an iterative process : iterations are called sprints and last in theory 30 calendar days. In practice, iterations usually last between 2 and 4 weeks. Each sprint has a goal and is associated with a list of items of product backlog (functionalities) to be realized. These items are broken down by the team in basic tasks of a few hours.

— Release : To improve the legibility of the project, iterations are generally grouped into releases. Indeed, as each sprint must lead to the delivery of a product a partial release allows to mark the delivery of a successful, likely version can be putted into operation.
— The Daily : Allow team to make a progress update on tasks and difficulties met. Each member takes turns answering three questions :
1. What did I do yesterday ?
2. What do I plan to do today ?
3. What difficulties do I encounter ?

3.2 Project time-line

So our project is divided into four major sprints : Each one of this sprints will be the subject of an entire chapter, were we will try to explain the different steps to achieve it.

- Requirement, Need specifications and General Design
- Set-up of dev environment : Yocto Project
- Qt and GUI App
- CAN and Diag App

ID	Sprint	Start	Finish	Duration
1	Diagnostic solutions, Analyse of requirement	2019-03-18	2019-04-12	20d
2	Yocto Project	2019-04-15	2019-05-22	28d
3	Dev env + Qt + CC	2019-05-16	2019-06-14	22d
4	CAN communication with ECU	2019-07-09	2019-07-22	10d
5	GUI App	2019-06-06	2019-07-08	23d
6	d-Bus IPC GUIApp + DiagApp	2019-07-25	2019-08-26	23d
7	Report	2019-05-20	2019-08-30	75d



Figure I.11 – Scrum methodology cycle

4 General Architecture

Here, we will highlight some particular features of BDiag solution. Besides, we will expose the general architecture of the project.

4.1 Presentation of BDiag solution

Before going further, we would like to enumerate some distinctly features that the BDiag solution will include and they aren't present in the previous solutions that we exposed in the subsection 2.2 page 7.
In fact the BDiag solution will be :

— Totally integrated in the car itself, it will be a generic solution based on Linux OS that can be installed in every car, and will be located in the place of multimedia reader.
— BDiag solution will communicate with pre-installed ECUs via CAN which becoming a standard communication interface in every vehicle.
— As a diagnostic tool, it will gives a daily diagnostic report and of course on demand diagnostic, besides live sensor data monitoring.
— Finally, the most distinctive feature is that our solution will gives real time assistance demo that helps users to fix their problems by self.

4.2 BDiag Architecture

As it is illustrated in the next figure I.12, the BDiag solution will be composed of four main part.

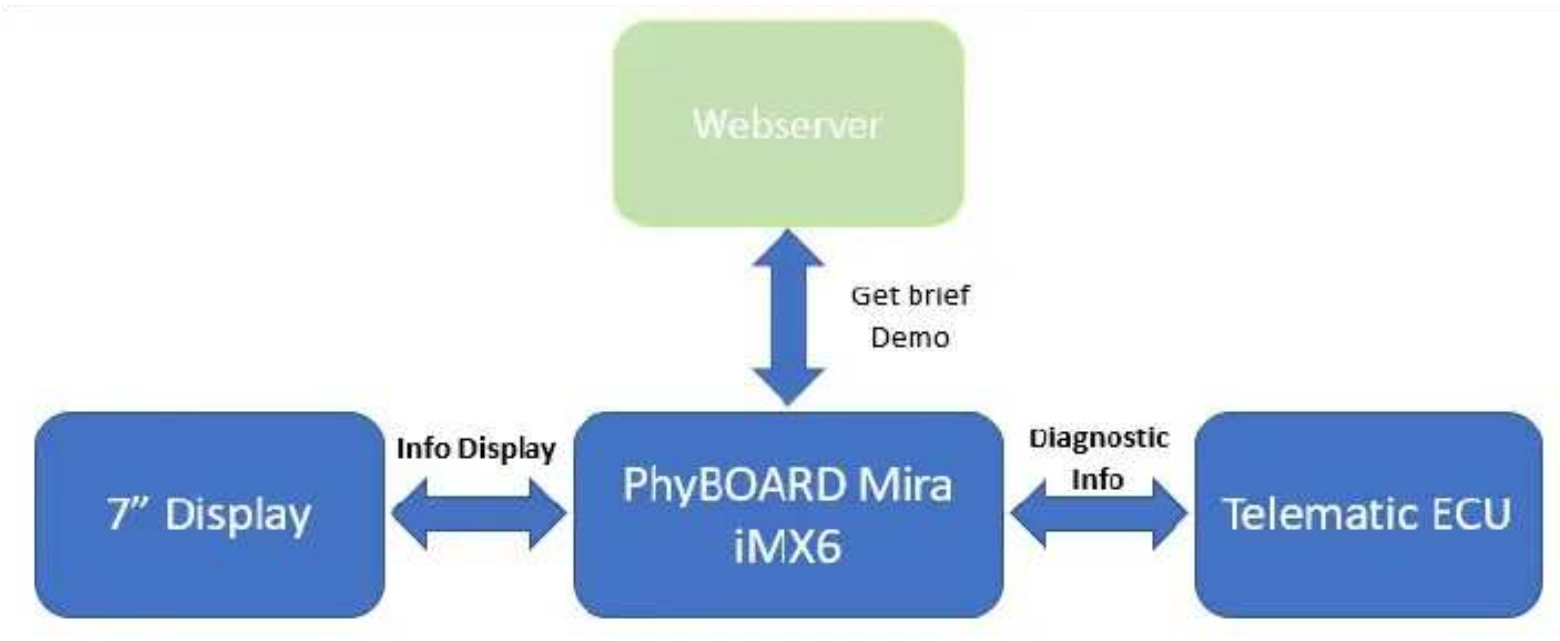


Figure I.12 – BDiag General Architecture

• First, the project will be based on the PHYTEC Mira iMX6 which is an embedded board running a Linux operating system and its dedicated specifically for automotive applications. This board will be the heart of the BDiag project, responsible for the communication with the other components to produce the desired functionalities.

• The telematic ECU, this ECU is devoted for handling internal and external communications whithen the vehicle. Including GPS, Wifi hotspot, SOS Ecall, OTA, Driver monitoring and infotainment. In addition it provides diagnostics informations, that will be used by our application. Those diagnostic informations will be obtained via CAN communication between the PHYTEC Mira and the telematic ECU.

• The third part is the 7" touch screen display that will be the direct interface with the car users. It will show diagnostic informations as well as assistance demonstrations.

• The final part will be the Cloud server that will contain all the assistance demonstrations(Video, GIF, Images, text..).

This was the general architecture of the BDiag solution, in the next section we will go more in deep details of the solution by performing requirement and specification analysis.

5 Requirement Analysis and Specification

One of the most important steps in software development is the needs analysis. This is a crucial step in the project lifecycle. It is used to define the aspects of functional and non-functional requirements of the application.

5.1 Specification of functional requirements

Our application must meet the functional requirements listed below :

— **Establish connection with the ECU :**
The system must be able, above all, to establish a connection with the calculator, in another word it ?s to be able to change the fields of requests and responses trams with this last.

— **Get Diagnostic Info :**
The system must be able to send a specific tram to the ECU for the diagnostic request,

to get the response trams from the ECU and it must also translate the in trams into interpreted messages that the user can understand.

— **Management of existing tests :**
The application must give the user the possibility to consult the list of existing detected problems and perform the operations of adding, modifying, deleting, executing, or importing a specific test.

— **Clearing of defects :**
After the reading of the defects and repairing them, the user can erase the defects stored list in the calculator so that they are no longer included in the next diagnosis session.

— **Measuring parameters :**
The user can perform a "parameters measurements" to visualize the values of a few sensors (in real time) on the vehicle and therefore he decides if it's work correctly or not.

— **Authentication :**

The diagnostic App must include "authentication" function especially when it comes to apply some administrative modification (add delete users) and/or access to critical information (how to demo). The authentication data will be stored in secured file or database.

— **Connect to the Internet :**
The system must be able to establish an internet connection with the internet, exactly with a dedicated server were many useful demos will be stored in different forms (Audio, Video or GIF). This demo will help users to solve their problems.

5.2 Specification of Non-functional requirements

Non-functional needs are the needs that do not express features of the system but it plays an important role when it comes to enhance the global system quality and to improve the user experience UX.

— **A good response time :**
The application must provide an acceptable response time (from the moment when the user chose to do a diagnostic, until the display info)

— **Ergonomics user interface :**
The application must be easy to use and easy to handle. It must also provide help messages to guide the user during diagnostic sessions.

— **A minimum error rate :**
The application must have a minimum error rate since a wrong diagnostic info can conduct to a wrong interpretation so a bad manipulation of the car.

6 System specification and Requirement modelling

After the analysis stage, we opted for the UML solution to model the operation of our system as well as the main player, through the "Use Cases diagrams". Indeed, these diagrams are limited to the actual claims of users. They do not include solution implementation and do not form a functional inventory of the system. But it gives us a better understanding of the system.

A use case allows us to write the interaction between an actor and the system. Therefore, a means to collect and write needs of the actors of the system. In these diagrams we present the differences functions that could be available to the main (car conductor).
The first use case presented in the Figure I.13, Global Use Case of the BDiag solution, explains the different tasks and functionalities offered by the system to the car user.

Figure I.13 – Global Use Case of the BDiag solution

After exposing the different aspects of the system, the different functionalities will be broken down to level one of use case diagrams. Then, the chronology actions of each functionality will be written by a sequences diagram.

Diagnostic : As it's shown in the Figure I.14, when the car user chose the diagnostic tab this will prompt him to many options menu :
* AutoDiag option in which a repetitive diagnostic task will be done automatically for a set time laps and for a specific set of components that can be fixed from the beginning by the user.
* ManualDiag option is available also to allow user getting an immediate diagnostic info for a desired part.
* The system will be able to store all this diagnostic data and saved them in history file, so the user can access them at any time or clear them when the problem is solved.

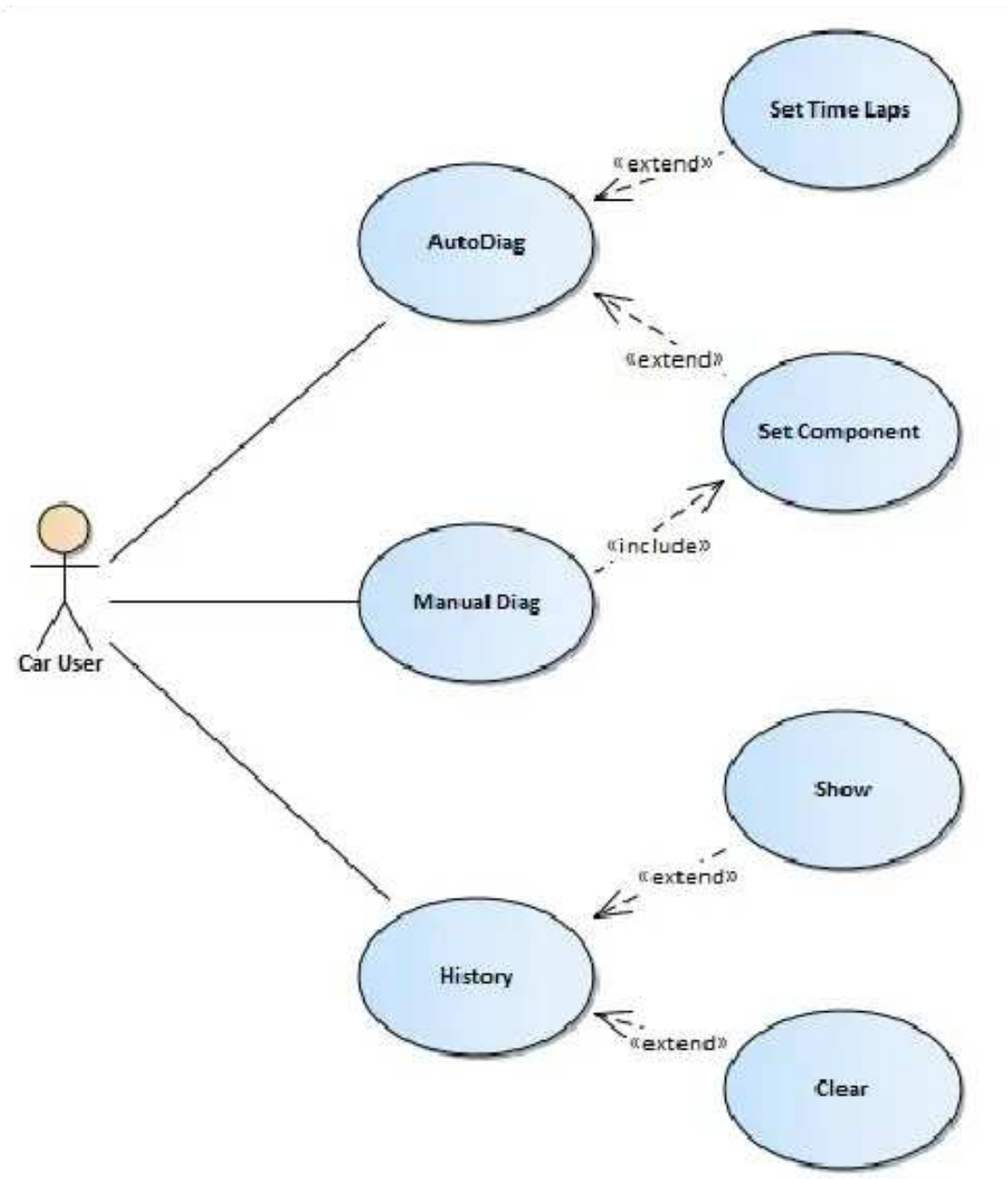


Figure I.14 – Diagnostic Use Case diagram

Connections : this is one of the very important part of BDiag system. In fact, after opening the application a set of connections must be established before the user can access to the different features as shown in the next figure I.15.

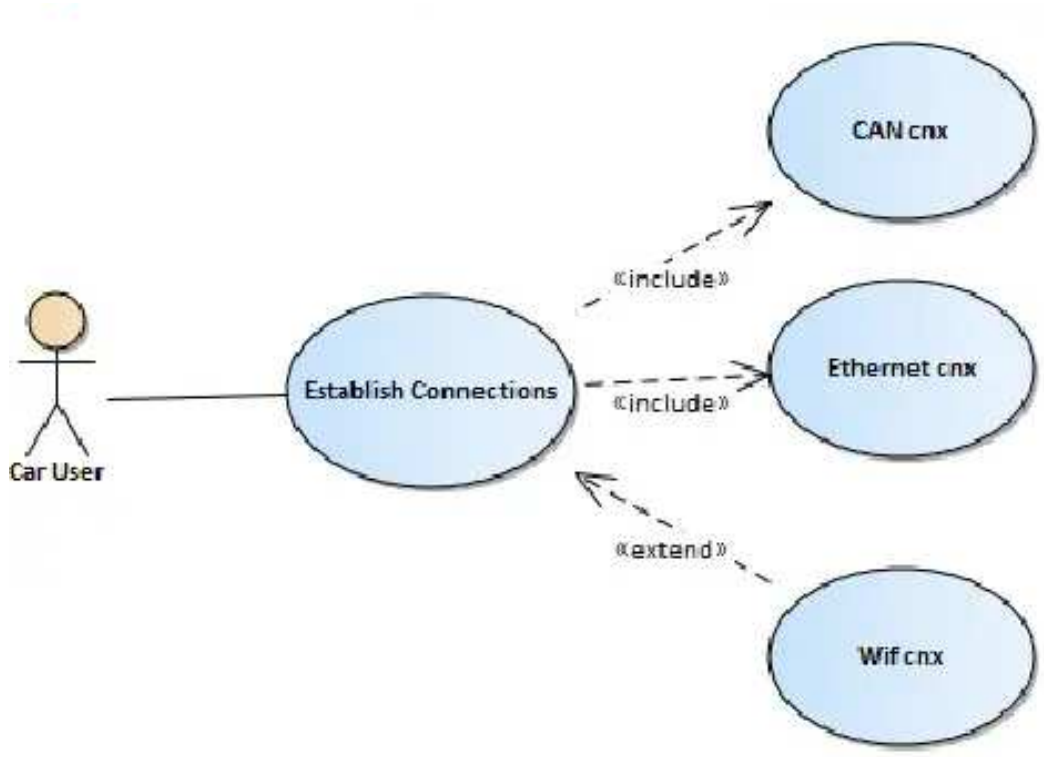


Figure I.15 – Connections Use Case diagram

First of all, while opening the application figure I.16. a connection request must be send from the GUIApp to the DiagApp. After that, the DiagApp will try to connect with the VCM and the Cloud Server in a strict order as shown in figure I.17

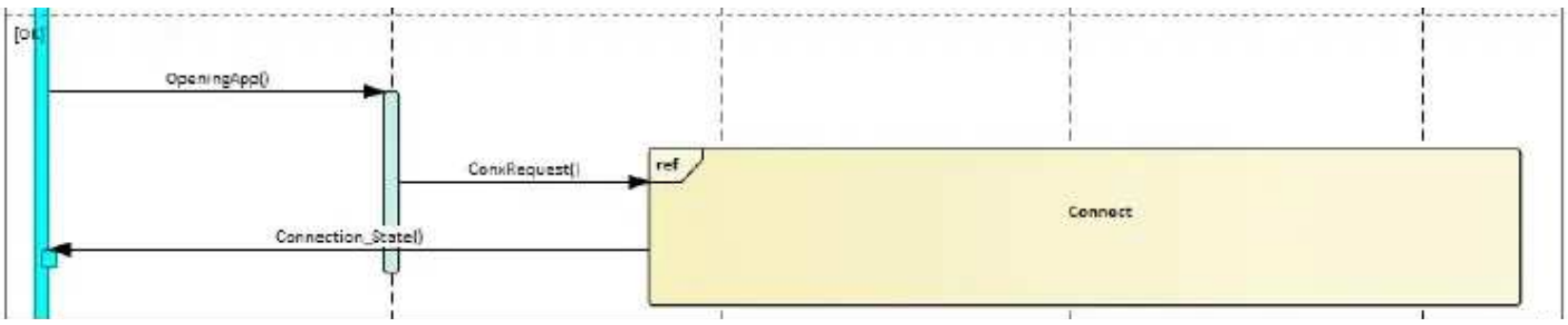


Figure I.16 – Sequence Diagram : Opening

— First with the ECU using CAN protocol. This connection request until it will be done successfully, or an Error message will appear on the screen after exceeded the Time Out delay.

— Second, when the connection with the ECU was established successfully, a connection request will be sent to the Cloud Server using Ethernet protocol (a WiFi protocol can be another option if it is available).

If this connection succeed a message will appear on the screen for some seconds and the user will have an access to the support Demo features. if not, an Error notification is shown on the screen for some seconds and the user will have no access to the support Demo features.

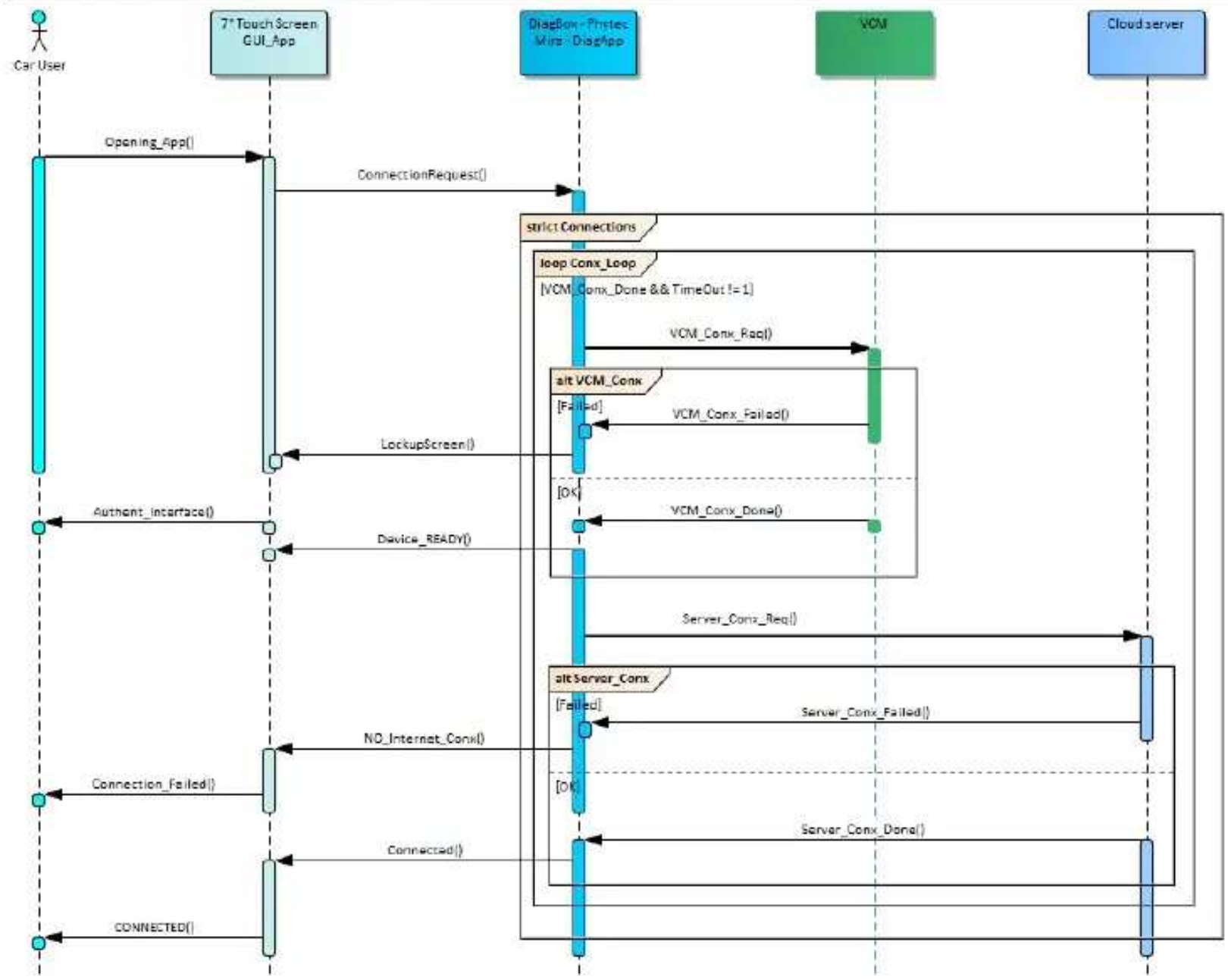


Figure I.17 – Sub Sequence diagram : Connect

Configurations : this part will gives the driver a better user experience, in which he can change the language, add or delete users and hopefully we will give OTA updates of the software in the future based on the user's feedback figure I.18.

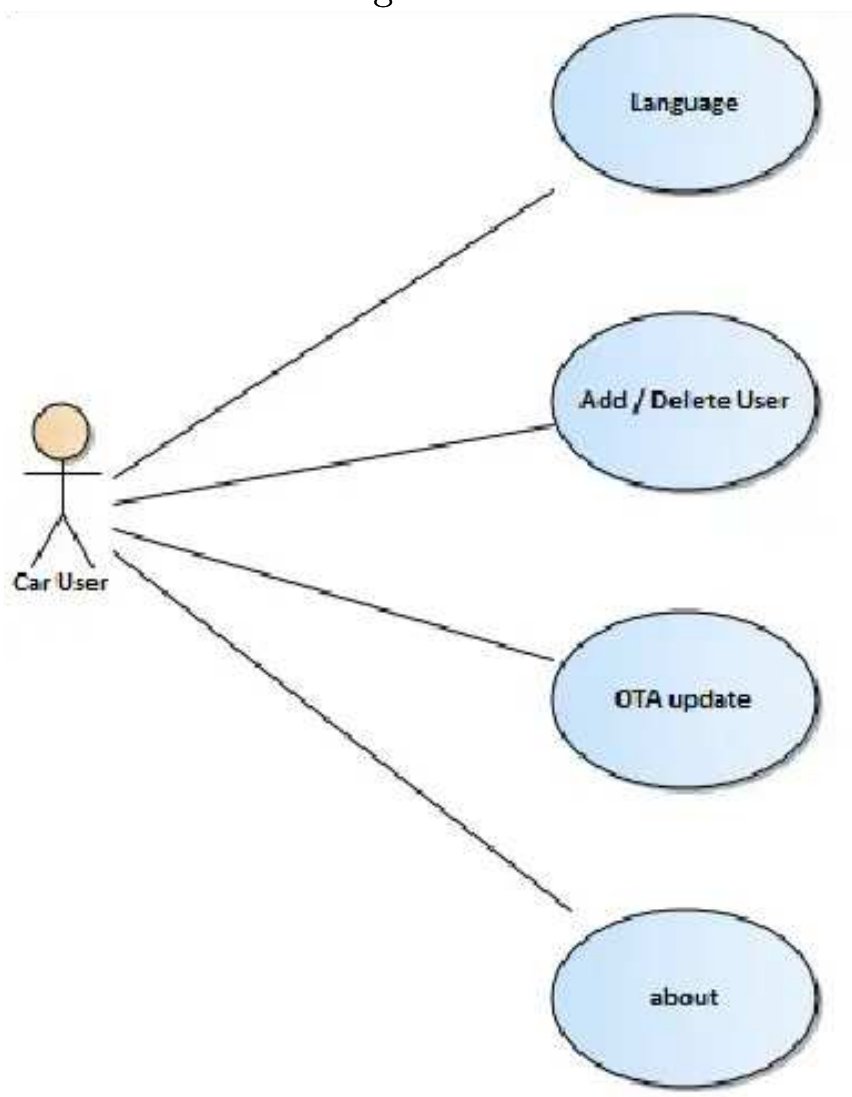


Figure I.18 – Use Case Diagram : Configurations

Conclusion

In this first chapter we introduced the general project framework, the importance of car diagnostic, especially with all the development in the automotive sector. Also we introduced the basic working methodology that we will adopt during this project.
In addition, we have developed a clear vision of the global solution. that's throw a deep analysis and need identification, as well as modelling of some specification using UML diagram. Also, a mind mapping (Annexe 1) can be found at the end of this report that expose the different parts to achieve such project. thus parts will be the main subject of the next chapters.

Chapitre II

Yocto Project

Plan

1 Embedded Linux 22

1.1 Advantages of Embedded Linux 24

1.2 The four elements of Embedded Linux 24

1.3 Toolchains 25

2 Yocto project 27

2.1 Core Components 28

2.2 YP Development Environment 29

3 Specific image for the phyBOARD-Mira 30

3.1 Target device : phyBOARD-Mira i.MX6 30

3.2 Create embedded Linux image for the phyBOARD Mira 34

Introduction

In this chapter we will start the project, by working on the first part which is the PHYTEC Mira iMX6. As any others embedded board, Mira needs an operating system to work with which is Linux OS in our case. This last can be obtained either from the official website of the board vendor, or generated using some powerful tools, and this will be the main subject of this chapter.

1 Embedded Linux

First of all, even though the term **Embedded Linux** is largely used, there is no such thing as *Embedded Linux!*. There is no special version of the Linux kernel for embedded systems, it is just the mainline Linux kernel running on an embedded system.
That said, the term embedded Linux has widespread and common use. Therefore, it is used generally instead of "Linux on an embedded system," which is the more accurate phrasing.

II.1 Embedded Linux

The word embedded in the term embedded Linux is used to convey the presence of an embedded system (the PHYTEC Mira iMX6 in our case), a concept that can be loosely explained as some type of computing hardware with integrated software that was designed to be used for a specific application.

However, embedded systems have some distinctive characteristics compared with desktop PC :

- They are designed to have specific and dedicated applications.
- They are often have limited processing power, memory availability, and storage capabilities.
- They are generally part of a larger system that may be linked to external sensors or/and actuators.
- They often have a task for which reliability is critical (e.g., controls in cars, airplanes, and medical equipment).
- They often work in real time, where their outputs are directly related to present inputs (e.g., control systems).

Embedded systems are present everywhere in everyday life figure II.1. Examples include vending machines, household appliances, phones/smartphones, manufacturing/ assembly lines, TVs, games consoles, cars (e.g., power steering and reversing sen-sors), network switches, routers, wireless access points, sound systems, medical monitoring equipment, printers, building access controls, parking meters, smart energy/water meters, watches, building tools, digital cameras, monitors, tablets, e-readers, anything robotic, smart card payment/access systems, and more.



Figure II.1 – Embedded systems

1.1 Advantages of Embedded Linux

There are many embedded platform types, each one has its advantages and disadvantages.

There are low-cost embedded platforms, with volume prices of less than 1 dollar, such as the (8/16-bit) Atmel AVR, Microchip PIC, and TI Stellaris, to high-cost specialized platforms that can cost more than 150 dollar, such as multicore digital signal processors (DSPs).

These platforms are typically programmed in C and/or assembly language, requiring that you have knowledge of the underlying systems architecture before you can develop useful applications.

Embedded Linux offers an alternative to these platforms, in that significant knowledge of the underlying architecture is not required to start building applications. However, if you want to interface with electronic modules or components, some such knowledge is required.

Here are some of the reasons why embedded Linux has seen such growth :

— Linux is an efficient and scalable operating system (OS), running on everything from low-cost consumer-oriented devices to expensive large- scale servers. It has evolved, over many years, from when computers were much less powerful than today, but it has retained many of the efficiencies.

— A huge number of open source programs and tools have already been developed that can be readily deployed in an embedded application. For example if you need a web server for your embedded application, you can install the same one that you might use on a Linux server.

— There is excellent open source support for many different peripherals and devices, from network adapters to displays.

— It is open source and does not require extra fee for its use.

One downside of embedded Linux is that it is not ideal for real-time applications due to the OS overhead. Therefore, for high-precision, fast-response applications, such as analog signal processing, embedded Linux may not be the perfect solution. However, there is some patchs that we can applied to Linux kernel like ***Xenomai*** that aim to use Linux in a preemptive way, interrupting the OS whenever required to maintain a real-time process [9].

1.2 The four elements of Embedded Linux

Every project begins by obtaining, customizing, and deploying these four elements : the toolchain, the bootloader, the kernel, and the root filesystem.

Bootloader : The program that initializes the board and loads the Linux kernel.

Kernel : This is the heart of the system, managing system resources and interfacing with hardware.

Root filesystem : Contains the libraries and programs that are run once the kernel has completed its initialization.

Toolchain : The compiler and other tools needed to create code for your target device. Everything else depends on the toolchain.

During this project most of the work will be based on the Cross-Toolchain. so, next we will ignore the other elements and we will focus only on the Toolchain.

1.3 Toolchains

A toolchain is the set of tools that compiles source code into executables that can run on specific target device. It includes a compiler, a linker, and run-time libraries. Initially we need toolchain to build the other three elements of an embedded Linux system : the bootloader, the kernel, and the root filesystem figure II.2. It has to be able to compile code written in assembly, C, and C++ since these are the languages used in the base open source packages.

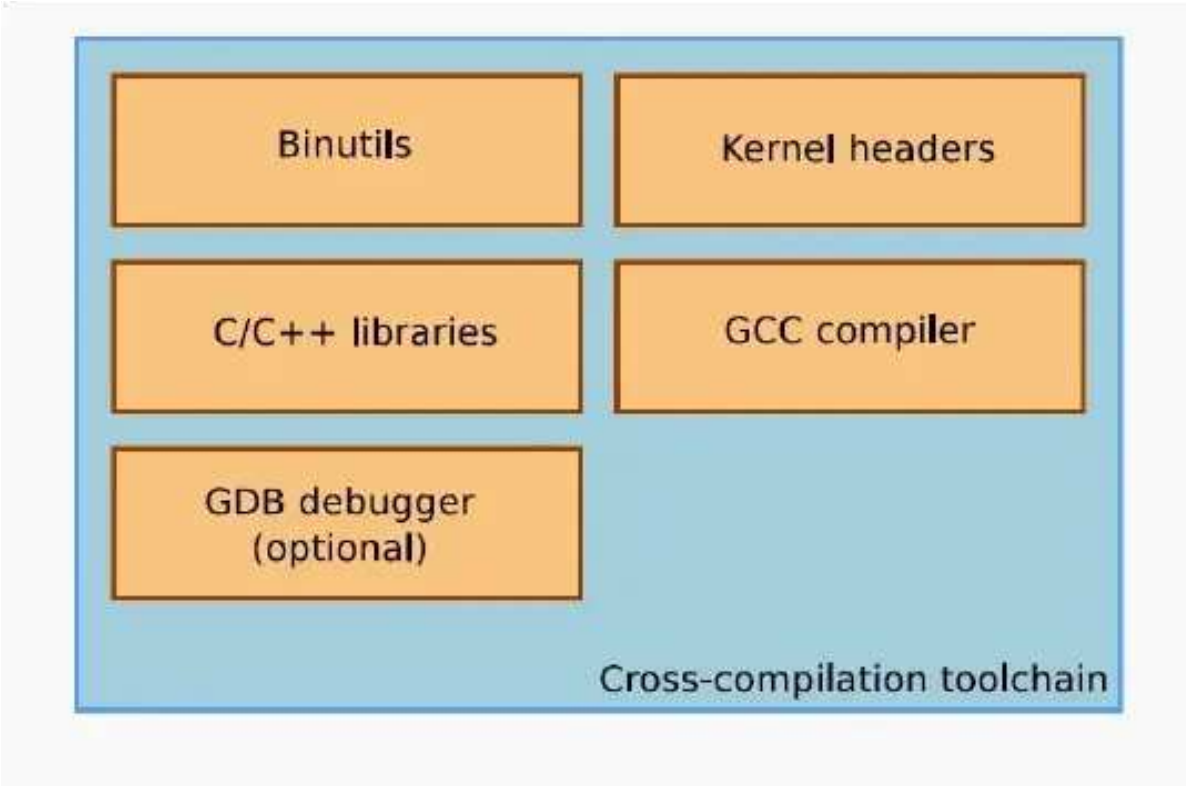


Figure II.2 – Toolchain components

Usually, toolchains for Linux are based on components from the GNU project ref [10]. A standard GNU toolchain consists of four main components :

- **Binutils :** A set of binary utilities including the assembler and the linker.
- **C library :** standardized application program interface (API) based on the POSIX specification, which is the main interface to the operating system kernel for applications.
- **GNU Compiler Collection(GCC) :** These are the compilers for C and other languages which, depending on the version of GCC, include C++, Objective-C, Objective-C++, Java, Fortran, Ada, and Go. They all use a common backend which produces assembler code, which is fed to the GNU assembler.
- **GNU Debugger (GDB) :** The debugger on GNU/Linux, available for most embedded architectures. it can be used to control the execution of a program, set breakpoints or change internal variables. You can also use it to see what a program was doing when it crashed (by loading its memory image, dumped into a core file). [11]

1.3.1 Types of toolchain

For our purposes, there are two types of toolchain figure II.3 :

- **Native :** This toolchain runs on the same type of system (sometimes the same actual system) as the programs it generates. This is the usual case for desktops and servers, and it is becoming popular on certain classes of embedded devices.

- **Cross :** This toolchain runs on a different type of system than the target, allowing the development to be done on a fast desktop PC and then loaded onto the embedded target for testing.

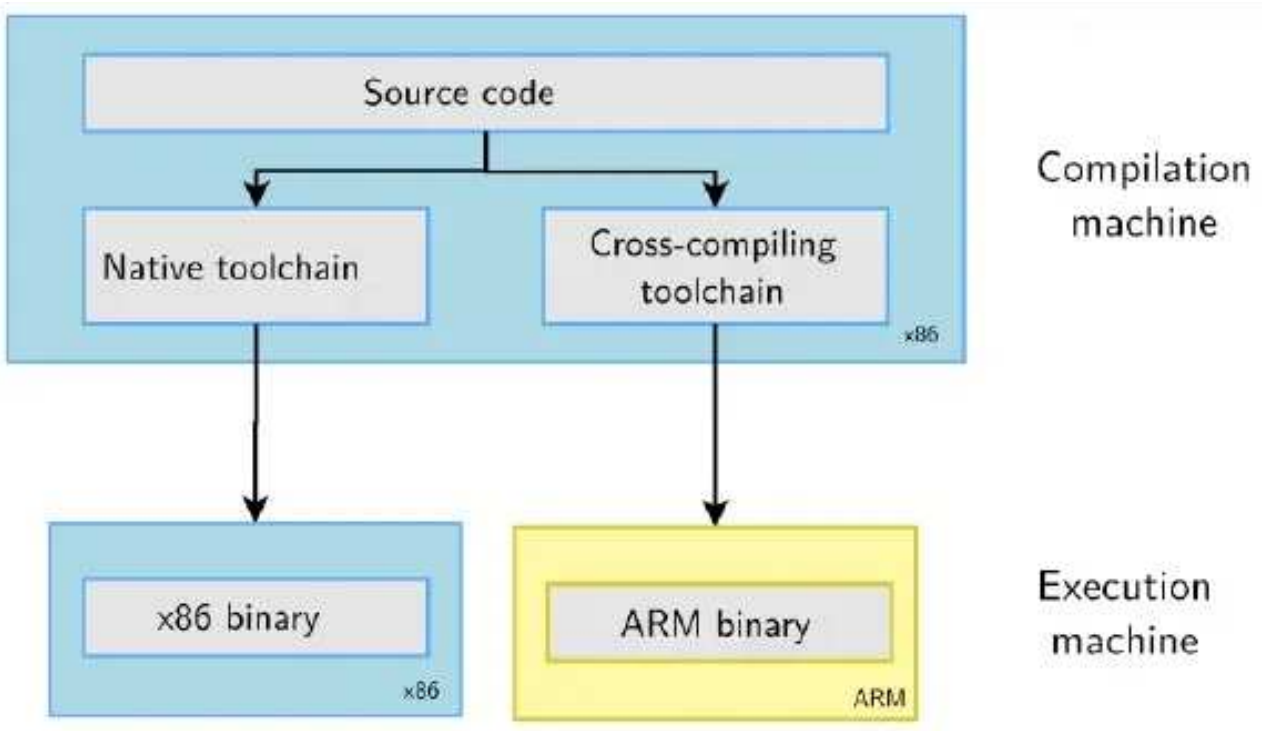


Figure II.3 – Toolchain types

1.3.2 Finding a toolchain

There is a few way to get a croos toolchain. it can be build either from scratch, creating each component sepertraly using step-by-step instructions from the CLFS project (Cross Linux From Scratch) [12].
- Or, a simpler alternative is to use **crosstool-NG**, which encapsulates the process into a set of scripts and has a menu-driven frontend. A fair degree of knowledge is needed to make the right choices.
- A more simpler way is the use of a build system such as **Buildroot** or the **Yocto Project**, since they generate a toolchain as part of the build process, And this will be the second part of this chapter, were we will talk about the **Yocto Project** consept before passing to the build process.
- Also, a cross toolchain can be optained directely from the chip-vendors like Arm that offers a GNU Arm Embedded Toolchain that can be downloaded from here [13].

2 Yocto project

Yocto is the smallest SI metric system prefix. Like m stands for Milli $= 10^{-3}$, so is yocto y $= 10^{-24}$. Yocto is also a project working group (figure II.4) of the Linux foundation and therefore backed up by several major companies in the field.



Figure II.4 – Yocto project logo

The Yocto Project is an open source collaboration project that provides templates, tools, and methods to help you create custom Linux-based systems for embedded products regardless of the hardware architecture.
It was founded in 2010 as a collaboration among many hardware manufacturers, open-source operating systems vendors, and electronics companies to bring some order to the chaos of embedded Linux development.

To be more preciously, like it described in the official website ref [14]

The Yocto Project.

It's not an embedded Linux Distribution,
It creates a custom one for you.

As said, the project wants to provide toolsets for embedded developers. It builds on top of the long-lasting OpenEmbedded project. It is not a Linux distribution. It contains the tools to create a Linux distribution specially fitted to the product requirements.
The most important step to bring order in the set of tools is to define a common versioning scheme and a reference system.

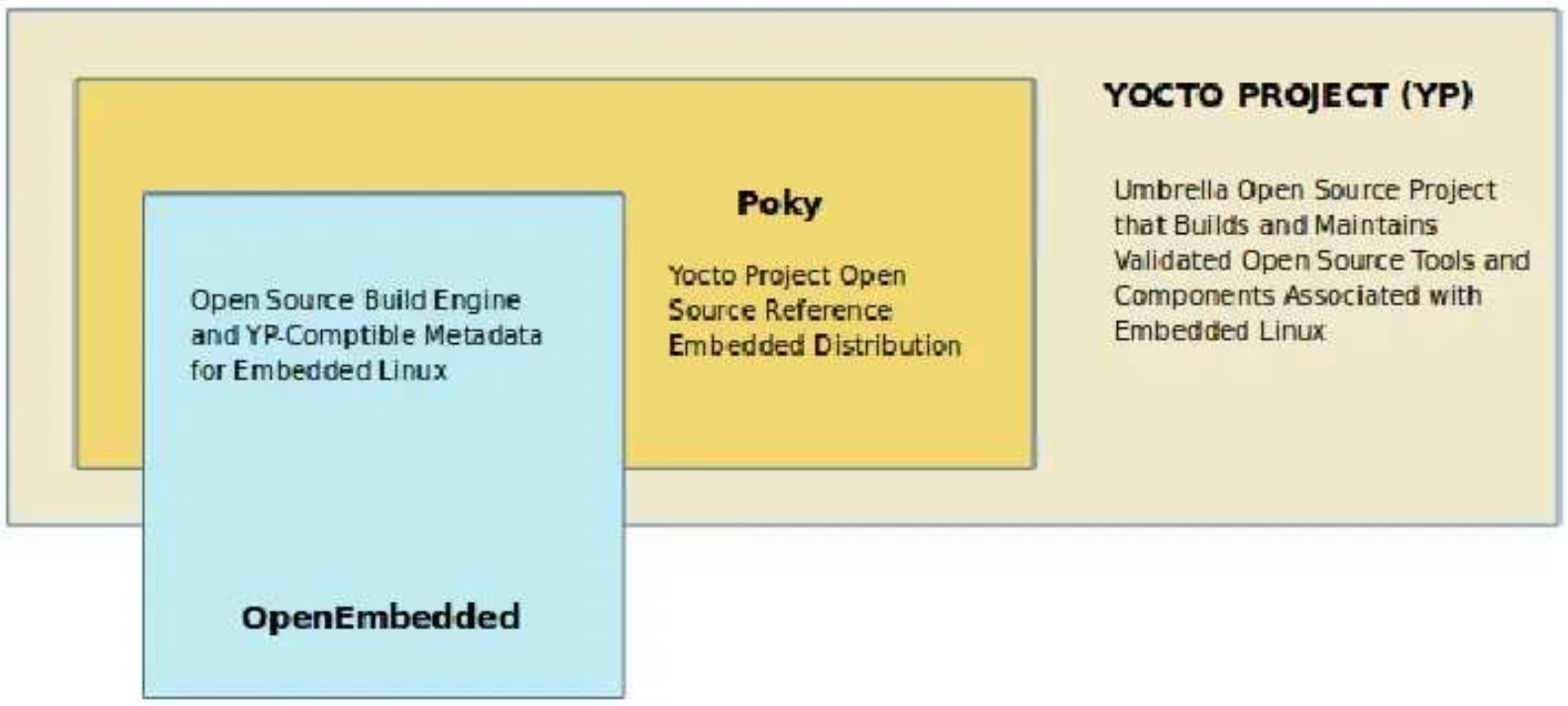


Figure II.5 – Yocto project origin

The project provides a flexible set of tools and a space where embedded developers worldwide can share technologies, software stacks, configurations, and best practices that can be used to create tailored Linux images for embedded and IOT devices, or anywhere a customized Linux OS is needed.

2.1 Core Components

The most important tools :

— **Bitbake** : is the build engine of YP, to be more clearer it a task scheduler like make, that interprets metadata a set of python scripts.

— **Recipes** : contain information about the software project (author, homepage, and license). defines dependencies, contains the URL of the source code, and describes how

to fetch, configure and compile the sources Classes : Classes combine functionality used inside recipes into reusable blocks.

— **Layers** : A layer is a collection of recipes, classes, and configuration metadata. A layer can depend on other layers and can be included or excluded one by one. It encapsulates a specific functionality and fulfills a specific purpose. Each layer falls into a specific category : Base Machine (BSP) Software Distribution Miscellaneous You can add one or many layers of each category in your build.

— **Machine** : it's a configuration variable which specifies the target hardware for example in our case : MACHINE?= "phyboard-mira-imx6-3".

2.2 YP Development Environment

The Yocto Project through the OpenEmbedded build system provides an open source development environment targeting the ARM, MIPS, PowerPC, and x86 architectures for a variety of platforms including x86-64 and emulated ones. You can use components from the Yocto Project to design, develop, build, debug, simulate, and test the complete software stack using Linux, the X Window System, GTK+ frameworks, and Qt frameworks.

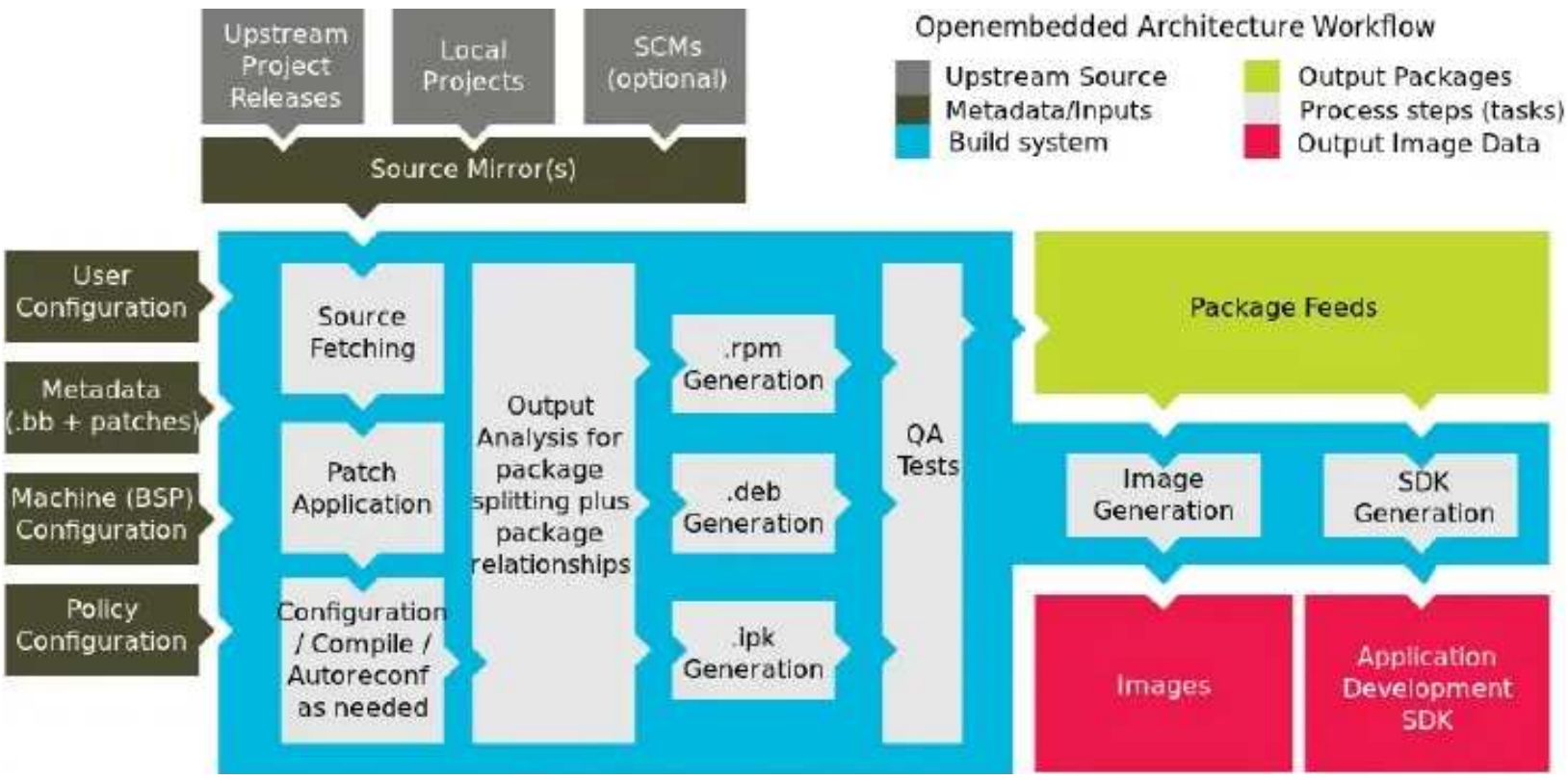


Figure II.6 – YP Architecture Workflow

Here are some highlights for the Yocto Project :
* Provides a recent Linux kernel along with a set of system commands and libraries suitable for the embedded environment.
* Makes available system components such as X11, GTK+, Qt, Clutter, and SDL (among others) so you can create a rich user experience on devices that have display hardware. For devices that do not have a display or where you wish to use alternative UI frameworks, these components need not be installed.
* Creates a focused and stable core compatible with the OpenEmbedded project with which you can easily and reliably build and develop.
* Fully supports a wide range of hardware and device emulation through the Quick EMUlator (QEMU).
* Provides a layer mechanism that allows you to easily extend the system, make customization, and keep them organized.

At this stage, we started the project, by working on the first part which is the PHYTEC phyBOARD Mira iMX6. But before going further, we would like to expose the general working environment and specifically the target board phyBOARD Mira iMX6.

3 Specific image for the phyBOARD-Mira

Before we describe the different steps to create Embedded Linux OS for our board, we would like to present deep technical details of our target board.

3.1 Target device : phyBOARD-Mira i.MX6

At this point it's time to introduce the heart of this project. In fact our embedded diagnostic solution is based on the **phyBOARD-Mira i.MX 6 Full Feature Kit** from PHYTEC company which is an industry-leading provider and integrator of System on Modules (SOM) and embedded middleware.

The phyBOARD-Mira is a high-performance single-board computer (SBC) for industrial applications. It designed to be compact and cost-optimized design, certifiable and industry-standard software support with Yocto-BSP.

II.3 Specific image for the phyBOARD-Mira

At the heart of the SBC is the phyCORE-i.MX 6 module figure II.7 with the NXP i.MX 6 Cortex-A9 processor.

It including some interesting features.ref [15], like :

- 1.0 GHz core clock frequency (optional 1.2 GHz).
- 1 GB (up to 2 GB2) DDR3 SDRAM.
- 1 GB (up to 16 GB2) on-board NAND Flash.
- Support I2C, SPI, CAN, PCIe, HDMI, SATA, JTAG and 10/100/1000 Mbit Ethernet interface.
- Improved interference safety achieved through multi-layer PCB technology and dedicated ground pins.

A full feature description can be found at the end of this report Annex 1.

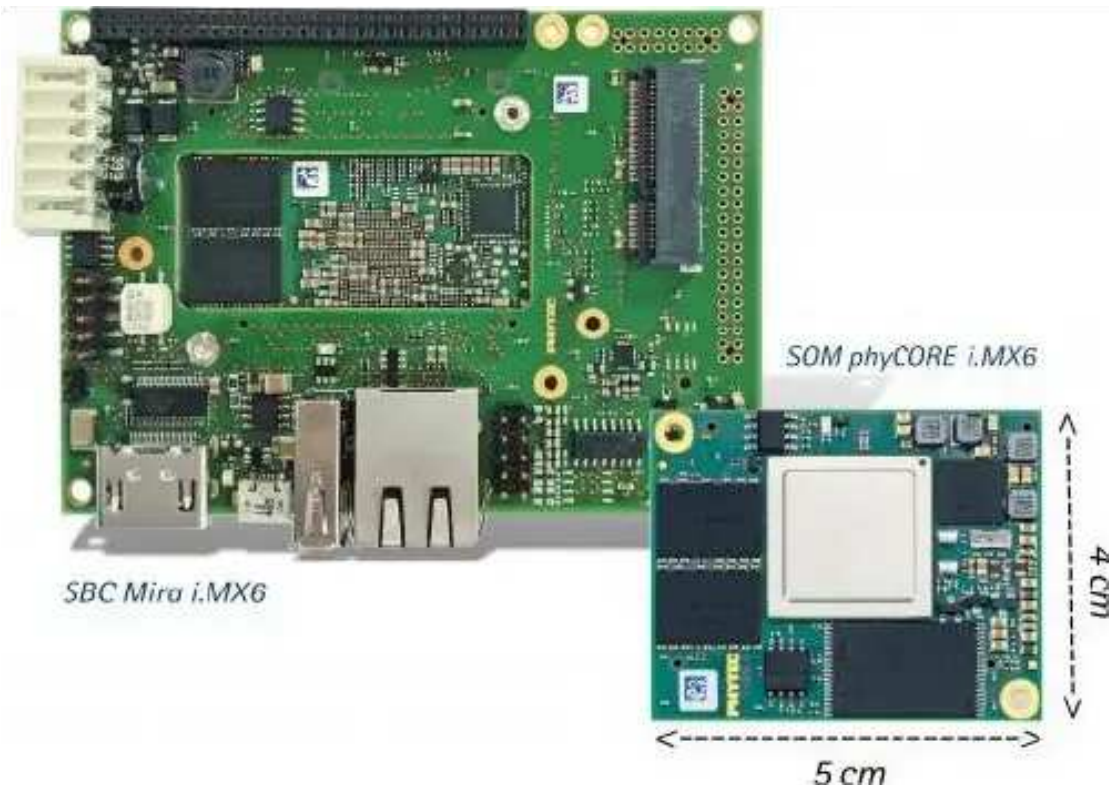


Figure II.7 – phyBOARD Mira i.MX 6 with phyCORE i.MX 6

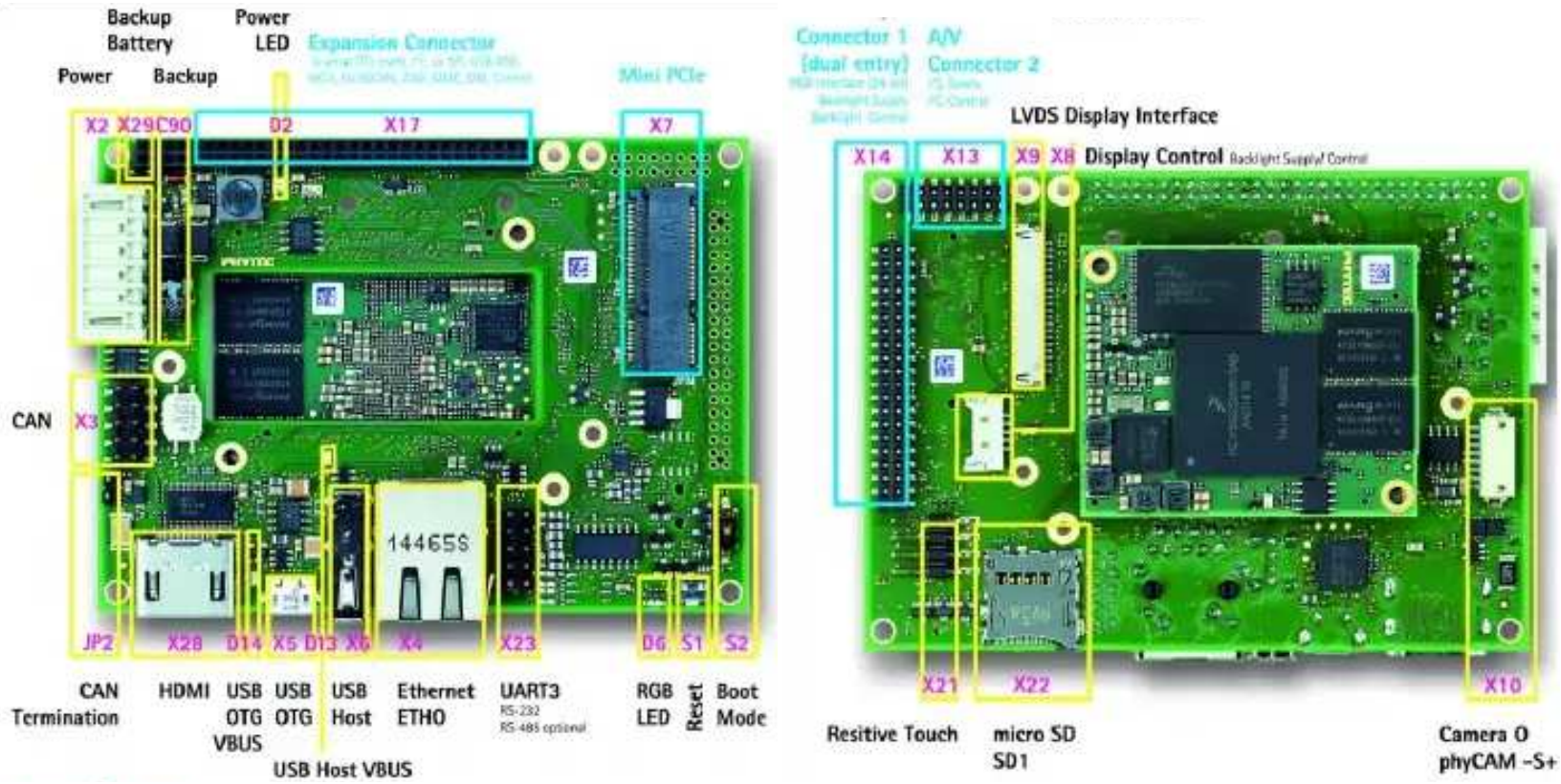
As well the phyBOARD-Mira [16] also presents some interesting features like :

- Max. 1.2 GHz core clock frequency and up to four cores.
- Expansion connector with different interfaces.
- One Secure Digital / Multi Media Memory Card interface brought out to a Micro-SD connector at the back side.
- Two different power supply options (5 V via 3.5 mm combicon or 12 V - 24 V through external power module).
- Backup battery supply for RTC with external 2-pole pin-header or with Gold cap (lasts

II.3 Specific image for the phyBOARD-Mira

approx. 11.5 days).

A full feature description can be found at the end of this report Annex 2.



(a) phyBOARD top-view **(b)** phyBOARD bottom-view

Figure II.8 – phyBOARD-Mira Connector Interfaces

Addition to the phyBOARD-Mira and the phyCORE-i.MX 6 the full kit that we have, includes other expansions that's needed to connect the phyBOARD with the external world.

- like the Evaluation module expansion (PEB-EVAL-01) figure II.9 which needed to provide a serial communication with the host machine through a dedicated serial cable included in the kit package.
- also, Audio/Video adapter module (PEB-AV-02) is needed to connect the board with the resistive touch LCD display.
- and finally, the power module (PEB-POW-01) to provide 12-24V input connector.

II.3 Specific image for the phyBOARD-Mira

Title	Accessory/Expansion Part Number	Description	Image
10.4" LVDS Display with Resistive Touch	AC116-T	10" AUO G104SN02-V2 LVDS Display with Resistive Touch	
Power Module (PEB-POW-01)	PEB-POW-01	Power module providing 12-24V input connector	
Evaluation Module (PEB-EVAL-01)	PEB-EVAL-01	Expansion board for Wega and Mira. Provides UART RS232, I2C EEPROM, and JTAG	
WiFi Adapter (PEB-WLBT-01)	PEB-WLBT-01	WiFi/Bluetooth module TiWi-BLE for phyBOARD-Mira	
AV Adapter (PEB-AV-02)	PEB-AV-02	LCD Display adapter module	

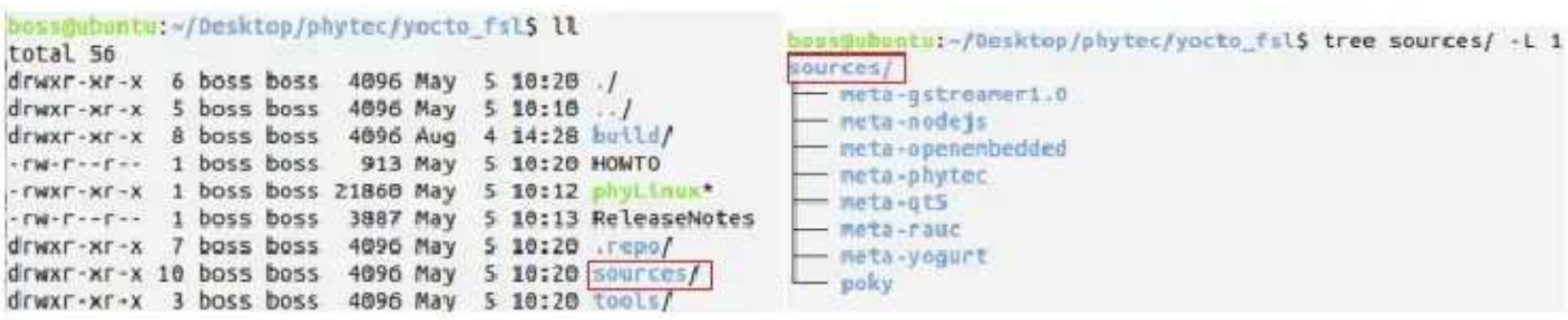
Figure II.9 – phyBOARD Mira Expansions

3.2 Create embedded Linux image for the phyBOARD Mira

It's time to start the build using Yocto 2.4.3 (rocko) and PHYTEC's Yocto BSPs. In fact, PHYTEC's Yocto BSPs are based on the Poky meta layer fig II.5, which consists of the bitbake build tool, Linux base recipes, and various scripts to define a rudimentary linux system.
The openEmbedded meta layer is also included in this BSP and is made up of a collection of meta layers which provide recipes for many software packages.
The meta-phytec layer contains recipes and classes common among all BSPs developed by PHYTEC. Within this layer is a SoC specific layer, such as meta-phyimx6, which defines configuration for barebox, the kernel, and software specific to the various PHYTEC boards using the i.MX6 ref [17].

Next we will highlight the most important steps to build the phyBOARD image with yocto project :

- First of all, we need to get the BSP meta-layers II.10, this can be done running the PHYTEC **phyLinux** script.



(a) YP working folder (b) Meta-layers

Figure II.10 – YP working directory

- After getting the meta-layers that we need, we run again the **phyLinux** script with **init** argument that guided us throw some steps to specify the exact model of our board, plus the desired version of image that we want to build and other information, like the support of the touch screen or not, etc...

- Now we are almost ready to start the build, but before going further, we need to specify other important parameters for the image *Global configuration* and the *Qt configuration* at the **local.conf** file Annex 3.

II.3 Specific image for the phyBOARD-Mira

In fact, we defined our host architecture for the cross-compiling toolchain "x86-64" for the 32/64 bits processors, also we added a light-weight editor "*nano*" as well as some useful packages for the Qt framework like qml-debug, qtquik1, qtquickcontrols and qtquickcontrols2. those packages are needed for the graphical interface.

```
MACHINE ?= "phyboard-mira-imx6-3"

DISTRO ?= "yogurt"

########################
# global configuration #
########################

# That are the default values of bitbake.  Adapt these to your workspace and
# host preferences.
DL_DIR = "/home/boss/Desktop/yocto_BDiag/yocto_dl"
#SSTATE_DIR = "${TOPDIR}/sstate-cache"

# License Handling
#  - Uncomment for i.MX6 proprietary GPU libraries
LICENSE_FLAGS_WHITELIST += "license-nxp_v14-june-2016_imx-gpu-viv"
#  - Uncomment for Freescale i.MX6 VPU firmware blobs
LICENSE_FLAGS_WHITELIST += "license-freescale_v12-march-2016_firmware-imx"

# build arttfacts are deployed here
DEPLOY_DIR = "${TOPDIR}/deploy"

# Select configuration UI for linux and barebox recipe. The openembedded
# default is 'menuconfig', 'nconfig' has more features.
# busybox only supports menuconfig
KCONFIG_CONFIG_COMMAND = "menuconfig"
KCONFIG_CONFIG_COMMAND = "nconfig"
KCONFIG_CONFIG_COMMAND_pn-busybox = "menuconfig"

# SDK will be build for that cpu
SDKMACHINE ?= "x86_64"
```

(a) Global configuration

```
####################
# QT configuration #
####################

# WE Disable qt widgets per default to save about 10 MB of storage. If
# you need qt widget support, you can enable it with this line
PACKAGECONFIG_append_pn-qtbase = " widgets gles2 qtgraphicaleffects-qmlplugins
qtquickcontrols2 qtquickcontrols2-mkspecs"

# qml-debug: This enables qml logging and profiling capabilities.
# It needs to be turned off for production use.
PACKAGECONFIG_append_pn-qtbase = " qml-debug"

IMAGE_INSTALL_append ="nano \
    qtgraphicaleffects-qmlplugins \
    qtquickcontrols-qmlplugins \
    qtquick1 \
    qtscript \
    "

PACKAGECONFIG_append_pn-qtbase = " accessibility"
PACKAGECONFIG_append_pn-qtquickcontrols = " accessibility"

IMAGE_INSTALL_append = " qtdeclarative-qmlplugins qtquickcontrols-qmlplugins"

CONF_VERSION = "1"
ACCEPT_FSL_EULA = "1"
```

(b) Qt configuration

Figure II.11 – Build configuration "local.conf"

- Finaly, we initialized the working environment by running the ***oe-init-build-env*** setup script to define the Yocto Project's build environment fig II.12 on our build host.

```
boss@BDiag-PC:~/Desktop/yocto_BDiag/yocto_fsl$ . sources/poky/oe-init-build-env

PHYTEC

Welcome to the PHYTEC BSP-Yocto buildsystem.

Before building for the first time, you should take a look in the main
configuration file 'conf/local.conf'. There you can tweak the build system
settings and license configuration. For example you may have to accept the
Freescale/NXP GPU and VPU licence for an i.MX6 board.

Tested build target for this build configuration is:

   $ bitbake phytec-qt5demo-image
```

Figure II.12 – YP environment setup

II.3 Specific image for the phyBOARD-Mira

We started the build by exucuting the ***bitbake*** command with a specific OS image target, which is *phytec-qt5demo-image* as it shows in figure II.12. Note that alternate Embedded Images are located in various meta layers at meta*/recipes*/images/*.bb.
They can be found using the command :
host$***bitbake-layers show-recipes "*-image*"*** in **build** folder.

Also, we must note, that the build process could take **several hours** the first time we run it. Even with our powerful host configuration (Intel Core i7-7700 CPU 8M Cache, up to 4.20 GHz, 4 cores and 8 threads, 16 GiB of DDR4 memory and Internet connection speed up to 200 Mbps), it takes about 4 to 7 hours depending on the internet speed, with bitbake tool configured to run the maximum simultaneous tasks fig II.13 depending on the available number of processors and cores.

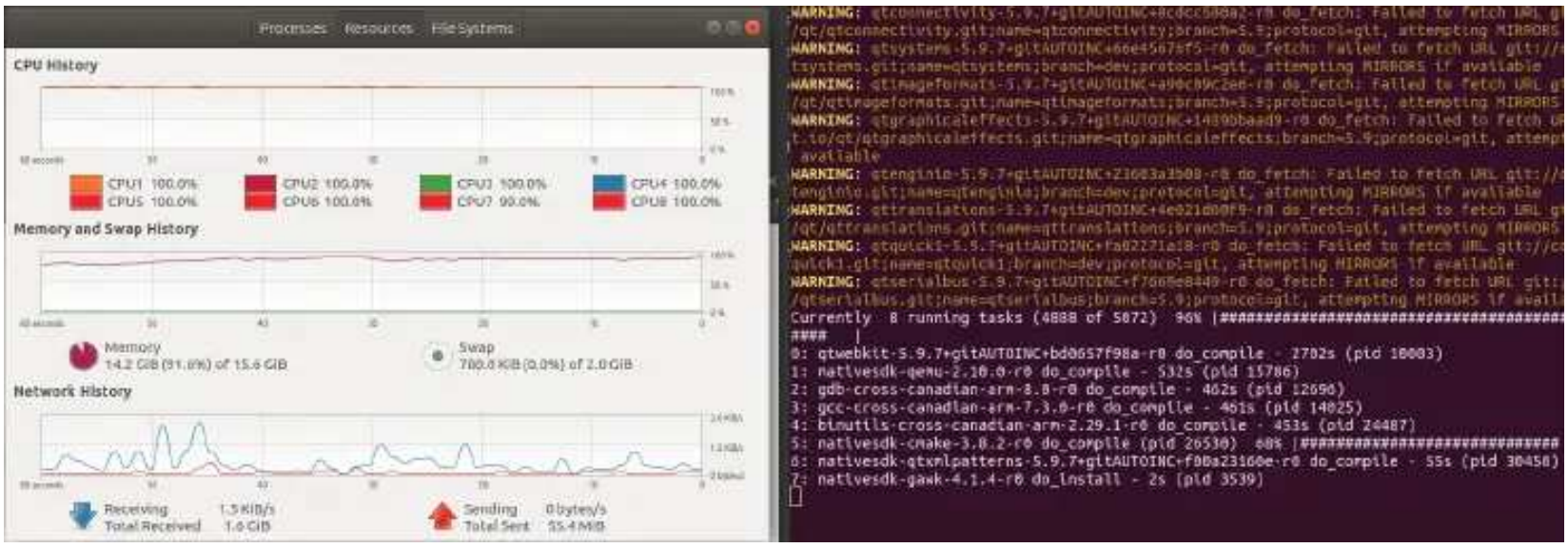


Figure II.13 – Bitbake CPU/RAM usage

Finally, there is some additional configuration that we need to set before that we get a successful build. In fact, during the build process, we encounter problems when bitbake tool fetching source code (e.g. fetcher failures or Git failures).
This issue is related to the proxy firewall used by ACTIA ES, and this painful problem was solved using "Working Behind a Network Proxy" page of the Yocto Project Wiki ref [18].

After many tries, we got a success build that generated an SD image phytec-qt5demo-image-phyboard-mira-imx6-3.sdcard (PD18.1.0) for the phyBOARD-Mira, a **barebox** Bootloader (v2017), **zImage** Kernel (v4.14 is a LTS version), and **phytec-qt5demo-image-phyboard-mira-imx6-3** Root Filesystem.

After that we executed $***bitbake phytec-qt5demo-image -c populate sdk*** to generate a cross-toolchain for our intel 64bit host architecture.

Next we will describe how we boot up the board for the first time, how we activate the different expansions and how we setup the cross-toolchain with Qt creator.

Conclusion

In this second chapter, we introduced Embedded Linux, as well, we enumerated some of its advantages, and we explained why embedded Linux becoming a standard in everyday life systems. Also, we talked about the four elements of embedded Linux focusing on the toolchain which is an important element for our project.

We devoted a second part to introduce Yocto Project. Then we exposed the working environment including our target device phyBOARD Mira. Finally, we stared the project by creating a customized image for our board that feats our needs. Moreover, a cross-toolchain was build to work in more efficient way.

At this point, we prepared every thing to start the development phase. We successfully created our first image for the phyBOARD Mira iMX6 and we are ready to boot-up the board for the first time.

Chapitre III

Implementation of BDiag Solution

Plan

1 BDiag user interface : "BDiagGUI" 38

1.1 Connect with the Mira Serial / SSH 39

1.2 Qt : Graphical development environment 40

1.3 Cross-Compiling Qt for phyBOARD-Mira 40

1.4 BDiagGUI interface 43

2 Using Git for BDiag solution 47

3 The engine of BDiag solution : "BDiagApp" 49

3.1 Inter-process communication with D-Bus 49

3.2 Controller Area Network for ECU communication 52

3.3 CAN Bus on the phyBOARD-Mira 53

Introduction

In previous chapter we prepared everything to start implementing the requirement specification and analysis that we did in the first chapter. This implementation will be based on two main applications :
- A GUI application we called "BDiagGUI" that will handle all the graphical staff.
- A diagnostic application we called "BDiagApp" that will handle the inter-process communication between the two application, besides the communication between the target board and the telematic ECU through CAN protocol, and finally, the diagnostic process itself.

1 BDiag user interface : "BDiagGUI"

This is the front-end layer of our embedded diagnostic solution and support for vehicles. Here we chosen to work with Qt cross-platform for many reasons that we will discuss later. But first, we need to add the cross-toolchain that we prepared before, and we need to setup a set of connections between the target and the host PC to get a successful cross-compilation.

1.1 Connect with the Mira Serial / SSH

Srial communication : After creating a bootable SD Card from the *phytec-qt5demo-image-phyboard-mira-imx6-3.sdcard* file, we selected the Boot Mode to boot from the sdcard with the S2 dipswitch on the Carrier Board figure II.8a.
At this point a Serial communication with the phyBOARD-Mira is needed, at least for the first boot. It's used to control the phyBOARD-Mira from the host PC through debug adapter figure III.1 and didcated terminal software (e.g. *Minicom*, or *Tera Term*).
First, we configured Minicom terminal for 115200 baud, 8 data bits, no parity, 1 stop bit (8n1) and no handshake. After booting the board, we are allowed to see everything happening on it even during the boot process. and this will be necessary to enable the display expansion in *barebox* environment by editing the *config-expansions* file.
Now, we have activated the 7" touch display and we have a full control on the phyBOARD-Mira but we still limited by a one terminal session. this will be a disadvantage when we would like to debug CAN frames (two terminal session will be needed : one for can-send and other for can-read frame). So another type of communication is required.

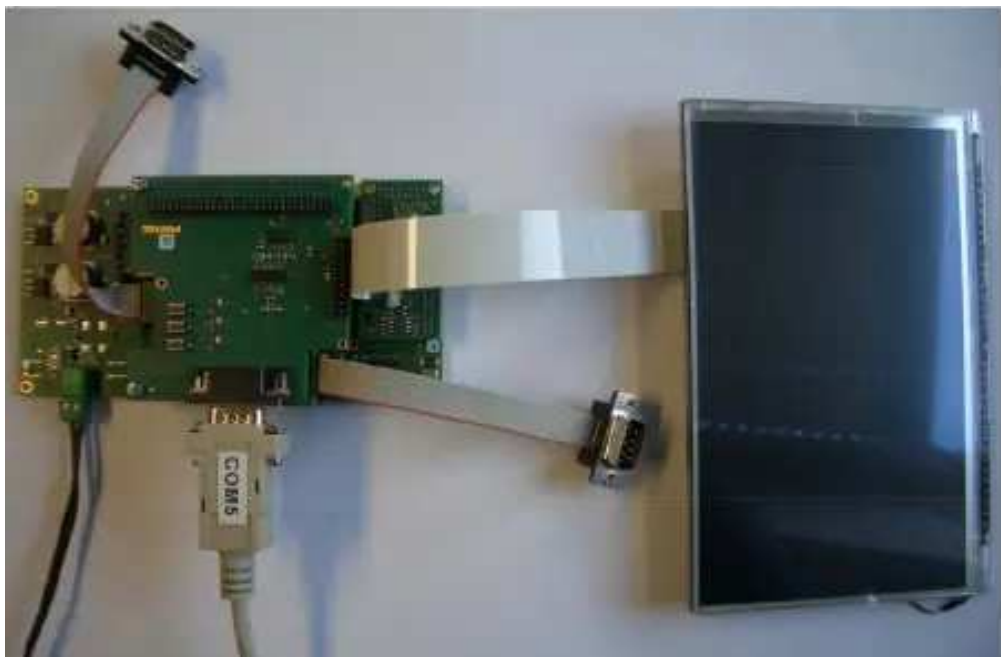


Figure III.1 – Serial comm

SSH communication : Secure Shell (SSH) is a useful network protocol for secure encrypted communication between network devices. We can use an SSH terminal client to connect our PC to the SSH server running on port 22 of the phyBOARD. This allows us to do the following :
- Execute commands from the host.
- Exchange files with the phyBOARD using the SSH File Transfer Protocol (SFTP).

- Forward X11 connections, which allows to perform virtual network computing.
- Deploy Qt applications to the phyBOARD-Mira from Qt creator.

1.2 Qt : Graphical development environment

Qt (pronounced "cute") III.2, is a powerful cross-platform development framework that uses standard C++. It provides libraries of C++ code for GUI application development and for database access, thread management, networking, and more. Importantly, code developed under this framework can be executed under Windows, Linux, Mac OS X, Android, iOS, and on embedded platforms, such as the phyBOARD-Mira. Qt can be used under open source or commercial terms and it is supported by freely available development tools, such as qmake and Qt Creator.



Figure III.2 – Qt logo

The capability and flexibility of this framework make it an ideal candidate for GUI applications that may running directly on the target device phyBOARD-Mira, or on host PCs.

1.3 Cross-Compiling Qt for phyBOARD-Mira

Cross-compilation (or cross-building) is the process of building software on one machine architecture that will execute on another completely different architecture. A common example would be building applications on an Intel-based workstation, which must execute on an ARM-, PowerPC-, or MIPS-based target device. Fortunately, the GNU tools make this process much less painful than it sounds[19].

Here we'll provide a quick overview of how to configure Qt creator to work with the cross-compiler that we have generated before. Next we will highliet the most importan steps, on how to add a kit, how to configure a project to work on the target and how to deploy it.

III.1 BDiag user interface : "BDiagGUI"

First of all, we started by configure an SSH connection for our target as showing in the next figure III.3

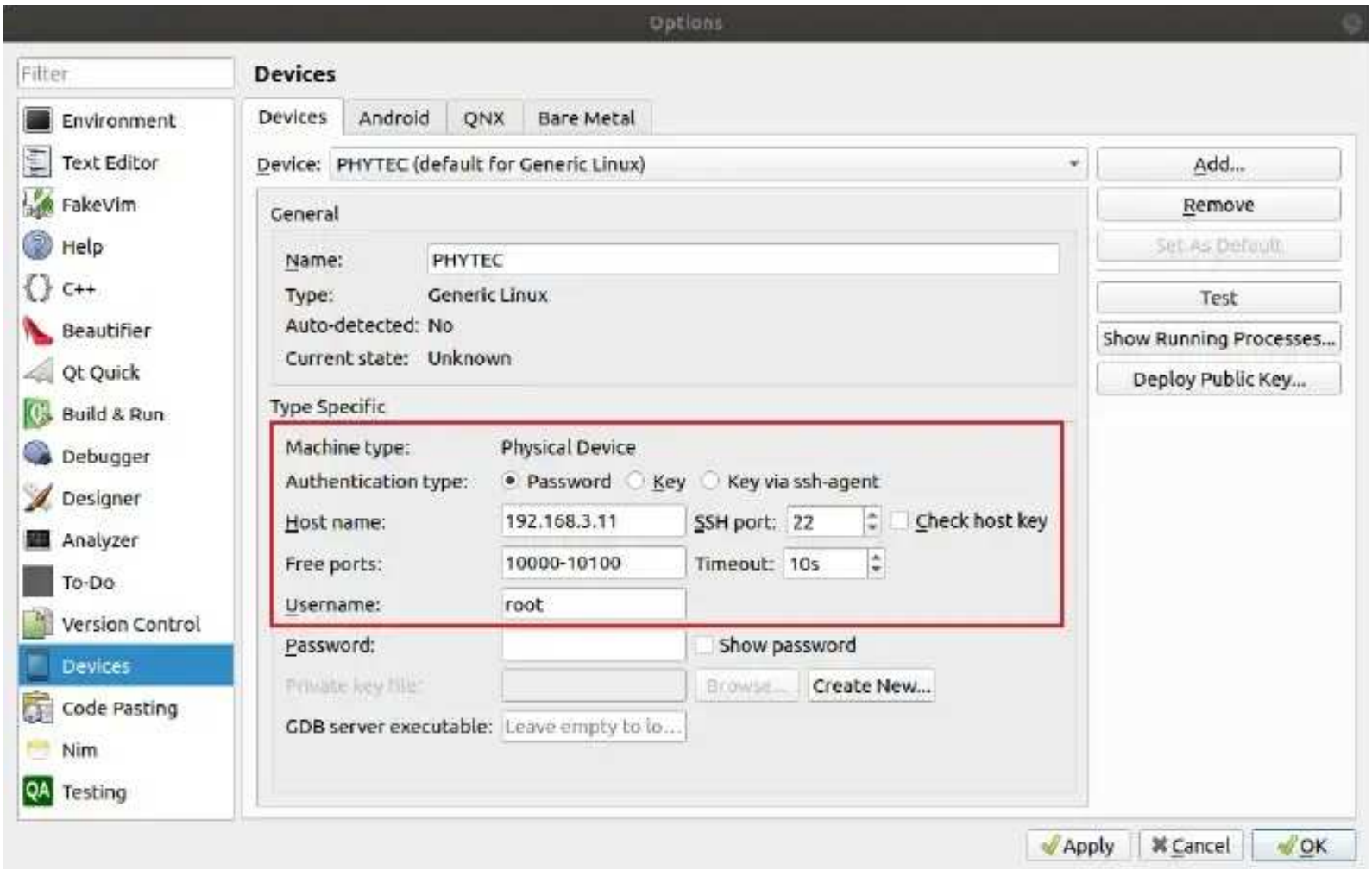


Figure III.3 – Qt SSH configuration

As a second step, we added the **arm-phytec-linux-gnueabi-gcc** and **arm-phytec-linux-gnueabi-g++** respectively for the C and C++ cross-compilers, from the SDK directory thet we created before using Yocto Project. The **GCC_18** name, act as a link for the long path of the C cross-compiler. the same thing for the **G++_18** and **GDB_18** figure III.4.
18 is for the version-number of the current embedded image that we worked on.

As well, we added a debugger **arm-phytec-linux-gnueabi-gdb** named **GDB_18**, so we can deploy the project into the target, likewise we can run and debug it from the host PC.

Finally we arranged all things together, we added some other information like the **Qt version** that our Embedded image was based on : (QT 5.9.6) besides the path of **Sysroot** filesystem directory.

Moreover, we created a new kit named **phyBOARD-Mira_18** as showing in the next figure III.4, that allowed us to cross-compiling our project directly into the phyBOARD-Mira.

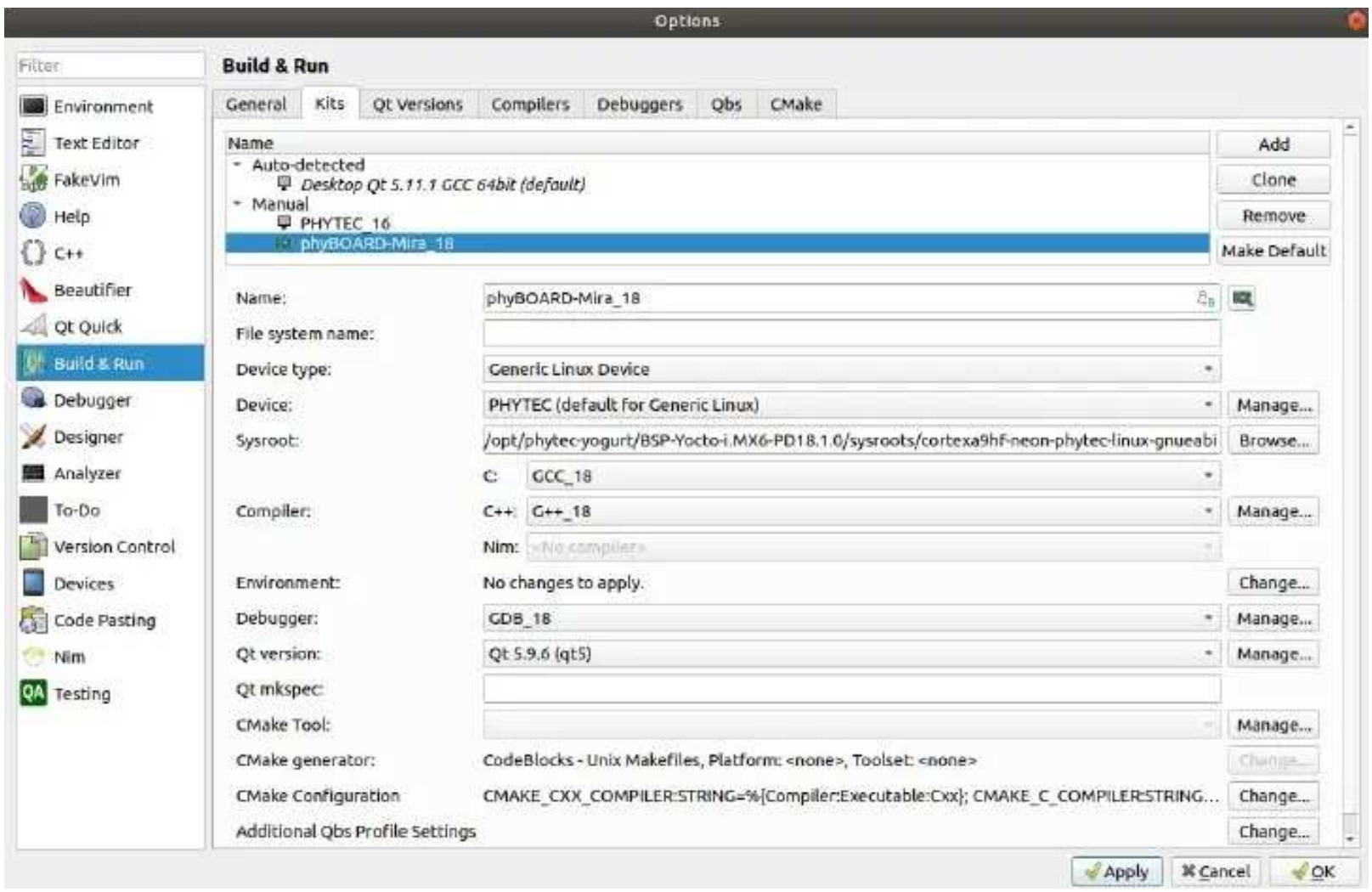


Figure III.4 – Qt development Kits

In case we select the phyBOARD-Mira Kit and we hit the Run button, a cross compiling process will start. If the build succeed, the executable binary will be sent to the embedded board and will be immediately executed in the target by the debugger.

At this stage we had a great cross-platform that allowed us to build and run the same source code into two different architectures. Besides the ability to build the code in the host PC and run it immediately into the target one. This will be a huge improvement in term of building and debugging time. Especially for persons how needs to see every code change impact as the case of graphical user interface developers.
So next we will discuss the development of the BDiagGUI application in addition, our choice for the "QML" language.

1.4 BDiagGUI interface

Before we expose our graphical part for the BDiag solution, we would like to give an overview of the used language which is "QML".

1.4.1 QML Syntax

Qt5 provides both native framework and QML which is Qt's replacement for HLTM5.
QML is a declarative language used to describe the user interface of your application. It breaks down the user interface into smaller elements, which can be combined into components.
QML describes the look and the behavior of these user interface elements. This user interface description can be enriched with JavaScript code to provide simple but also more complex logic. In this perspective, it follows the HTML-JavaScript pattern but QML is designed from the ground up to describe user interfaces, not text-documents. The next listing below III.1 is a good example that show this Philosophy.
Also QML consists of three major components : Qt Essential, Qt Add-Ons and Qt Tools. The package-group recipe is structured such that the whole user interface could be modularized.

Listing III.1 – Simple Button example with QML

```
Button {
    id: btON  // identification
    x: 38     // Horizontal position in the window
    y: 97     // Vertical position in the window
    text: qsTr("ON")    // printed text
    onClicked: {console.log("changed signal state ON")  // console message
                controller.turnOn()   // exucute turnoff methode on cliked}
  }
```

1.4.2 BDiag GUI App

Here, we will put some highlights on the implementation of the most important functionality (requirements) of our solution presented by the BDiagGUI application.
The next figure III.5 shows the homepage of the BDiagGUI solution. Where we can choose one tab from three options (Diagnostic, Configure and Info) by horizontal scrolling. Each one of these tabs contains a set of functions as following :

When we clicked on the *"Diagnostic"* tab, the next diagnostic page figure III.6 should appear on the touch screen. This page offers two types of diagnostic :

III.1 BDiag user interface : "BDiagGUI"

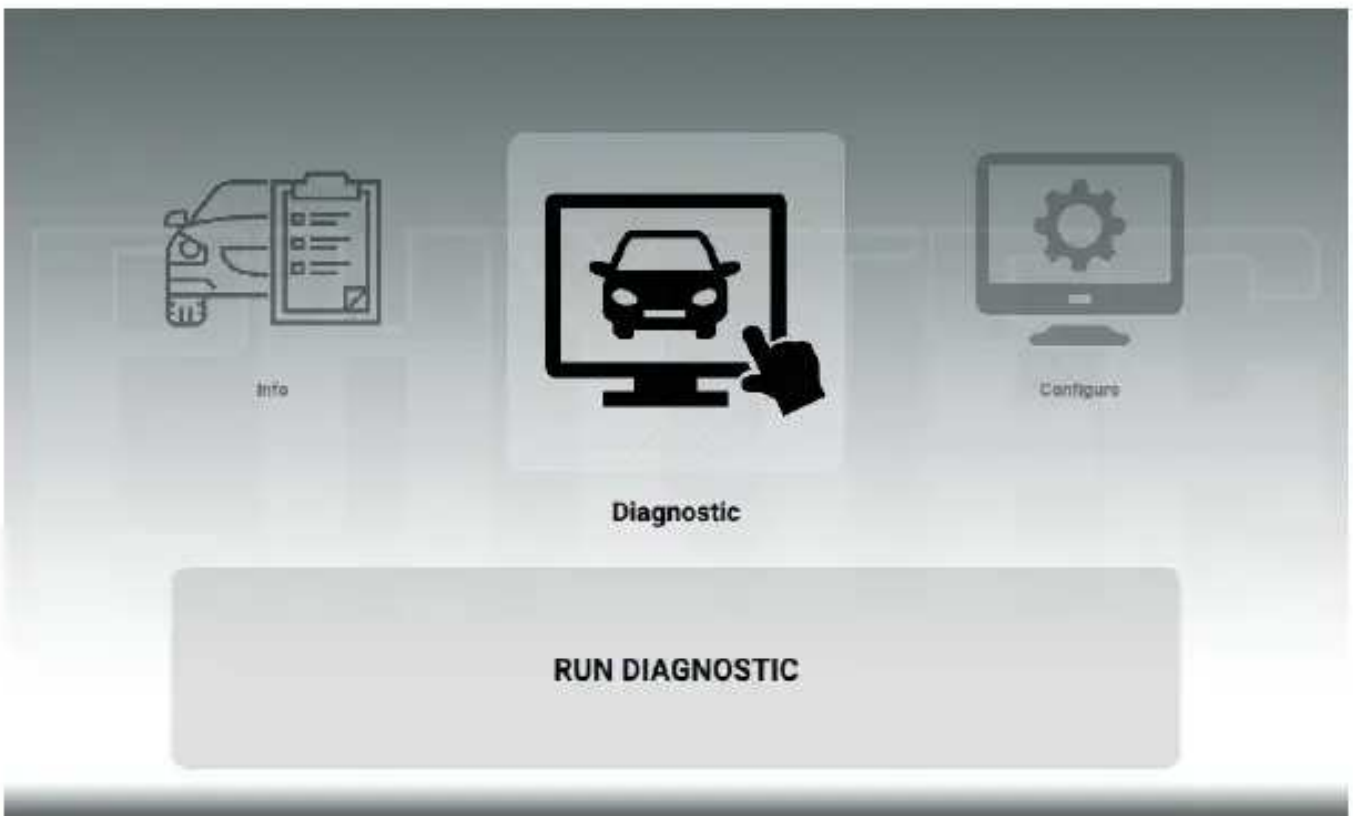


Figure III.5 – GUI main window

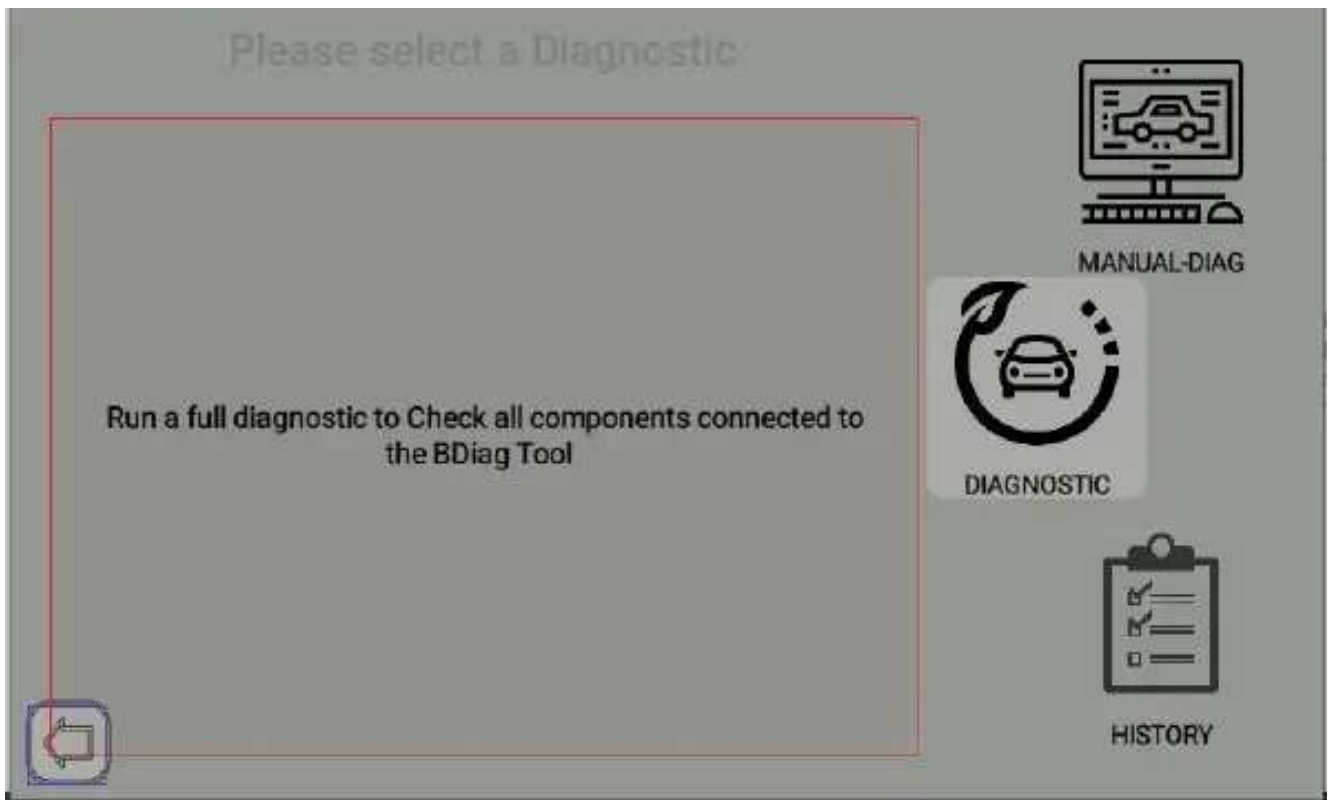


Figure III.6 – Diagnostic window

— DIAGNOSTIC : buy clicked on this option, a full diagnostic will running in the background. (we will give more details on this process, in the BDiagApp section).

This full diagnostic, will check every connected component and will shows a full diagnostic report as showing in the next figure III.7. This report shows the list of DTC and

a brief description for each one, besides the appropriate support for every fault if its available.

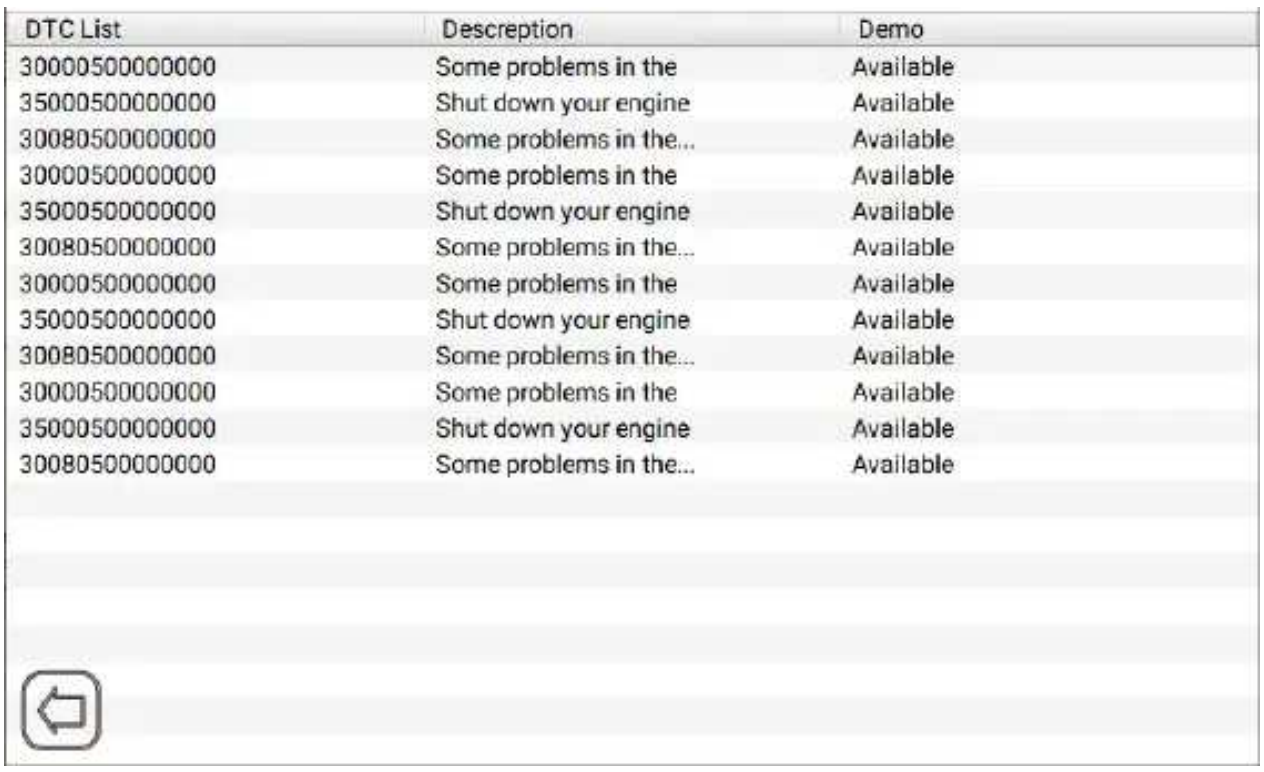


Figure III.7 – Full Diagnostic report window

— MANUAL-DIAG : in this tab figure III.8, the car user can select one or more component to be tested. At the moment when the user select a component a diagnostic request will be send from the GUI application to the diagnostic process BDiagApp.

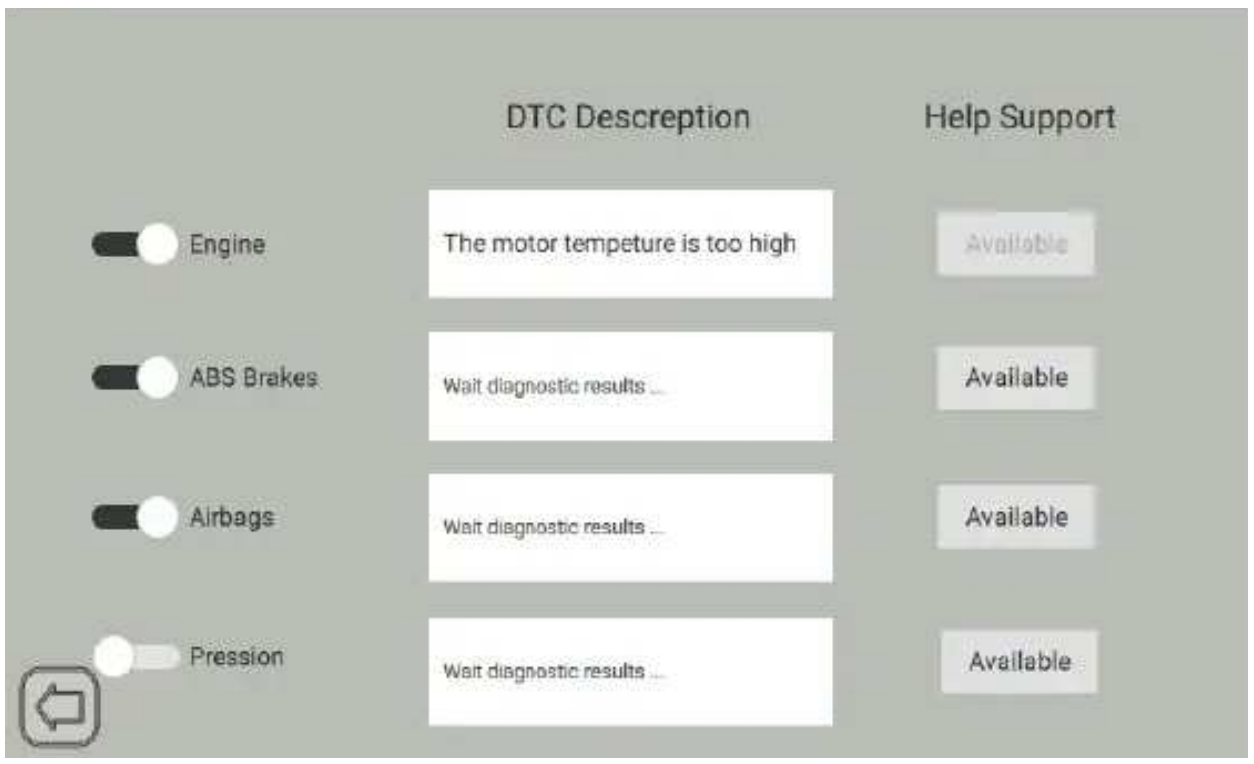


Figure III.8 – Manual Diagnostic window

The diagnostic result will printed in a specific case for each component. when a support demonstration is available, the right buttons will be in active mode so the user will have an access for the assisting support, else the buttons will not be clickable (inactive mode).

— HISTORY : After a diagnostic session either full or manual, the results will saved in another tab. so the user can access to the diagnostic results any time without running another test. Also, this tab will gives him the ability to erase the defaults code that he fixed.

We must note that : this previous diagnostic window was planed to be displayed in the red area in the main diagnostic window figure III.6. But for this first prototype will keep things simple. Also, we created a float "Back" button in the left corner, that appears in every window.

Back into the BDiaGUI main window, when selects the configuration tab, the next window will appear figure III.9. That allows the user to modify some parameters as the language, to verify the ECU connection and to do an OTA update.

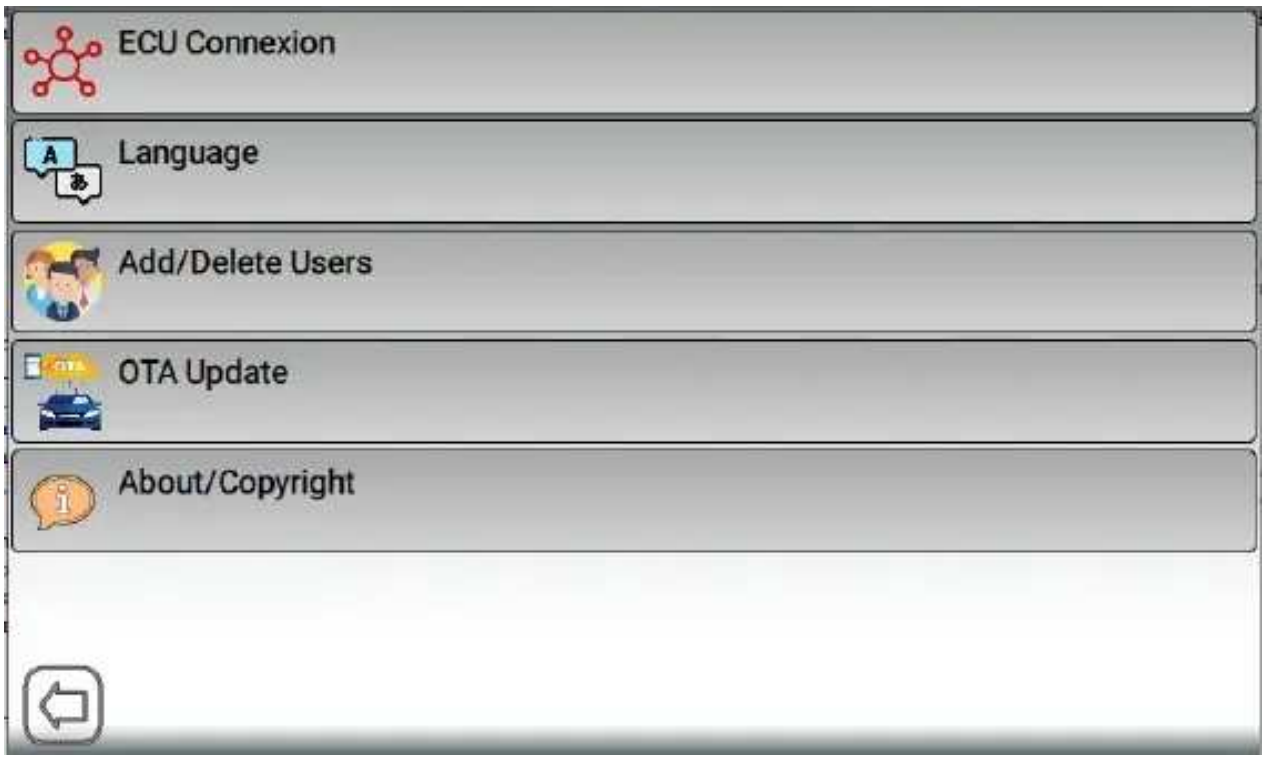


Figure III.9 – Configuration window

At this point, we created a graphical skeleton model for our BDiag solution. This skeleton is more than enough to implement the desired functionality at the background for this first prototype. There is no limit for what we can do with Qt. always we can enhance our GUI by combined the infinity items, animation and transitions offered by Qt.

So, we stopped at this level for the BDiagGUI and we passed away for the background process "BDiagApp".

2 Using Git for BDiag solution

Simply put, Git is a system that enables you to track changes to the content of a software project as it develops. Git, designed by Linus Torvalds, is used today for mainline Linux kernel development. Git is an incredibly useful system to understand for two main reasons : You can use Git when developing your own software, and you can gain an appreciation of how to work with Linux kernel source distributions.

Git is a distributed version control system (DVCS) for source control management.
A version control system (VCS) tracks and manages changes to documents of any type. Typically, documents that have been changed are marked with revision numbers and time stamps. It is possible to compare revisions and even revert to older versions of the documents.

Furthermore, Git can be used to manage a whole project by classified tasks into three main groups : ***Done***, ***In progress*** and ***To Do*** list, as showing in the next figure III.10.

This is a very efficient way to keep a clear vision during the project as well to maintain the SCRUM methodology during the different parts of it.

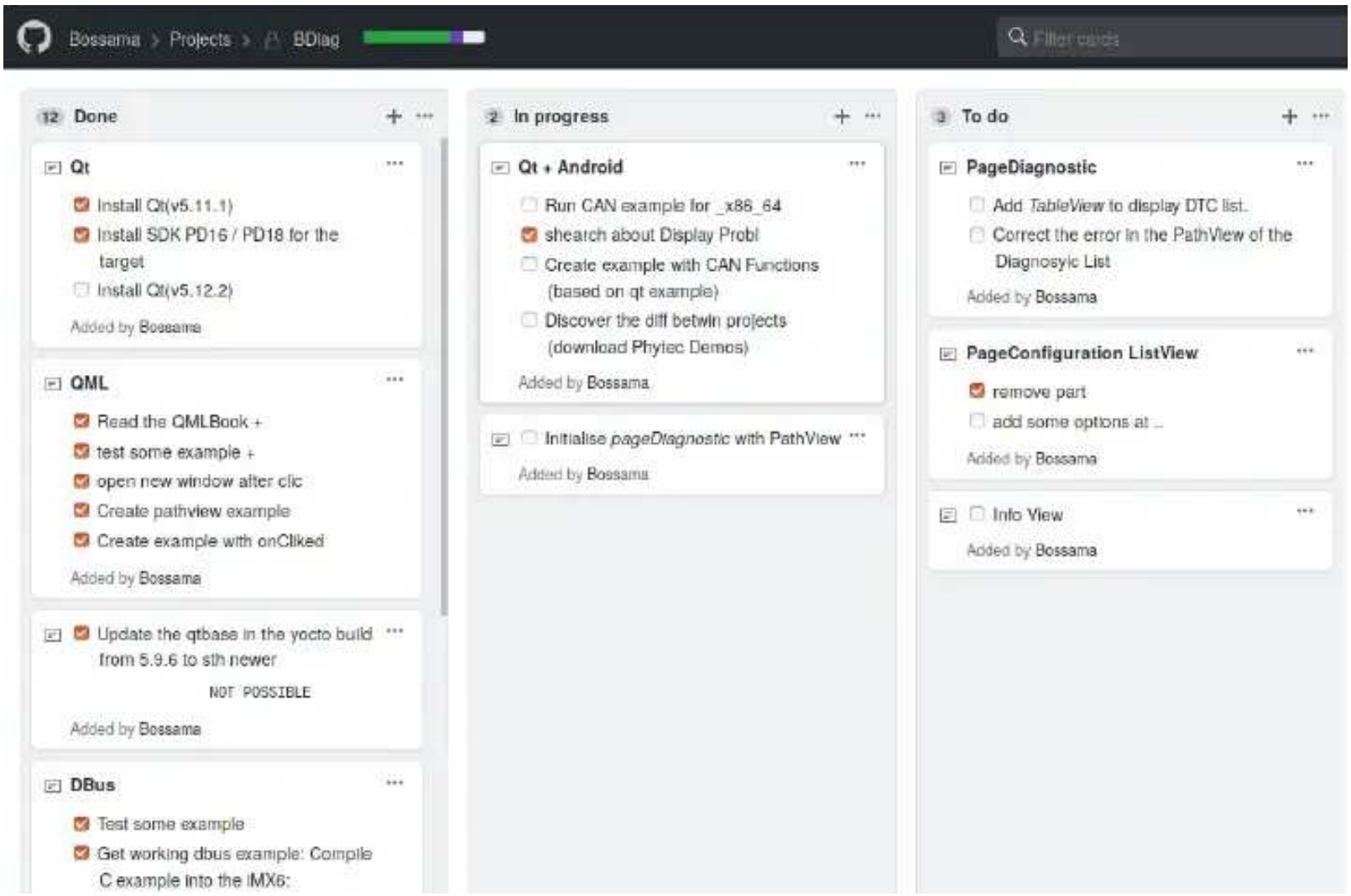


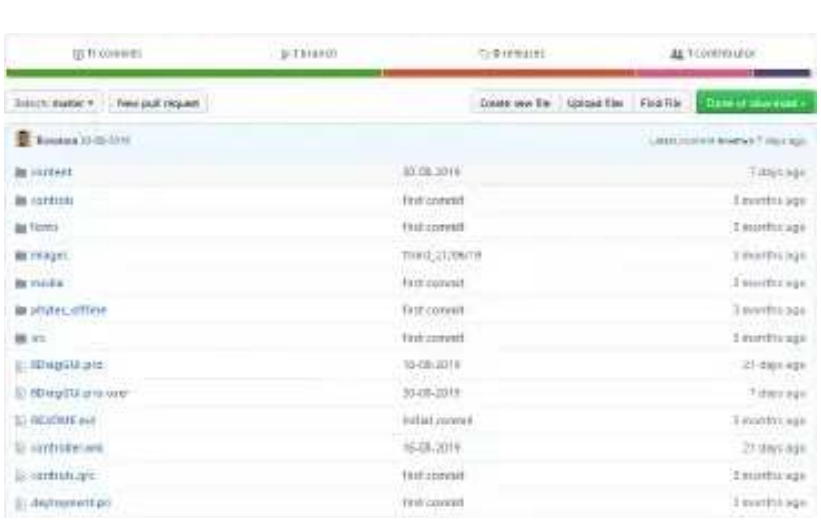
Figure III.10 – BDiag Git project view

III.2 Using Git for BDiag solution

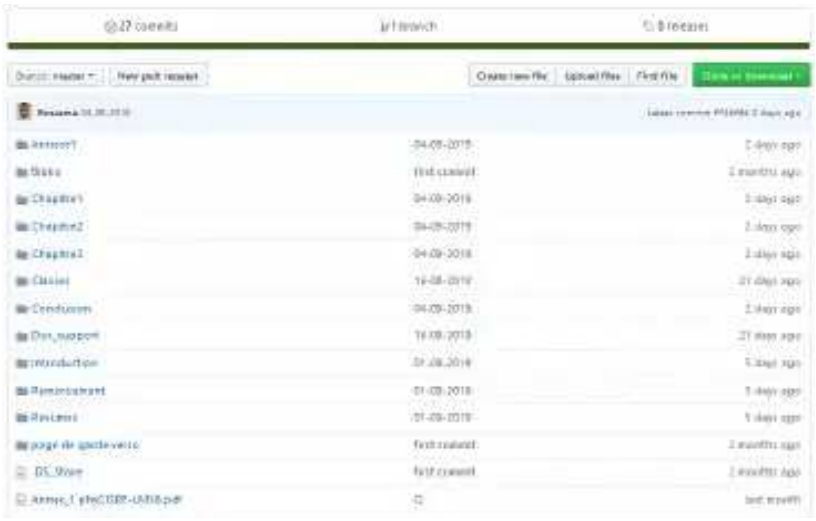
Of course, this project was linked with three others repositories next figure.

— **BDiagGUI** repository : for the graphical part of our solution.

— **BDiagApp** repository : for the diagnostic process and d-bus communication
— **BDiagReport** repository : for the report progress.



(a) BDiagGUI Git repository



(b) BDiagReport Git repository



(c) BDiagApp repository

Figure III.11 – BDiagApp repositories Git view

Typically, every time we made some meaningful changes on one of this repositories, we committed it. This can be done even with shell commands (Annex D) or by a graphical interface designed specially for windows users.

3 The engine of BDiag solution : "BDiagApp"

After preparing the development environment (Qt creator), creating the skeleton of the BDiagGUI and using Git to our work organized. Its time to pass through more serious things the BDiagApp.

3.1 Inter-process communication with D-Bus

D-Bus is an Inter-Process Communication (IPC) and Remote Procedure Calling (RPC) mechanism originally developed for Linux to replace existing and competing IPC solutions with one unified protocol. It has also been designed to allow communication between system-level processes (such as printer and hardware driver services) and normal user processes.
Also it is a service daemon that runs in the background. We used bus daemons to interact with diagnostic application (BDiagApp) and their functionalities as the graphical part (BDiagGUI). The bus daemon forwards and receives messages to and from diagnostic application. There are two types of bus daemons : SystemBus and SessionBus.
Before start doing any code, there are some terms that one should be familiar with :

- **DBusConnection** : is the structure to use for opening a connection to the daemon, either the session bus daemon by specifying DBUS_BUS_SESSION or to the system bus daemon using DBUS_BUS_SYSTEM.
- **Path** : Is the path of a remote Object, example /org/freedesktop/DBus.
- **Interface** : Is the interface on a given Object to communicate with.
- **DBus Message** : it's simply a message between two process, all the DBus intercommunication are done using DBus Message, these messages can have the following types,

  method calls, *method returns*, *signals*, and *errors*.
  The DBusMessage structure can hold parameters, by join strings, boolean, arrays, real numbers, integers,... to the message.
- **Method Call** : can be used to call a method on a remote object.
- **Signal** : It is a DBus message to deliver a signal emission.
- **DBus Error** : The DBusError is the structure that holds the error code which occurs by calling a DBus method.

Finally, we must note that the low-level API for DBUS is written in C but most of the documentation and code is written for a higher level binding, such as Python or GLib.
In this project we used both levels, the low-level API for the BDiagApp written in C and high-level API for the BDiagGUI.

3.1.1 D-Bus advantages

D Bus is optimum then the traditional IPC mechanisms , essentially because of its ease of use, and the extra features it provides. Just let list the most useful ones :

— D-Bus exist for several programming languages like C, C++, Java, Python, PHP etc. Either more, it offers, not only the inter process communication, but also the **inter language** communication too.

— D-Bus has a *broadcast* messages ability, so this allows the communication with multiple process at the same time,this will be helpful when we want multiple applications to respond to an event.

— D-Bus has a dedicated daemon (D-Bus Daemon), that can wake up desired services at the needed time. So we will avoid process that is always running in the background.

3.1.2 D-Bus on BDiag solution

Now, we will talk about how we used D-Bus for our solution to set a communicate between the BDiagGUI and BDiagApp. Before anything, we must ensure that the service process is running in the background. So for that, we created a service file named **BDiagApp.service**, that contain the name and the Exec key for the **BDiagApp** process. After that we placed the BDiagApp.service file under **/usr/share/dbus-1/services**. In this way, whenever we restart or power on the system, our service will executed automatically at the boot time.
Then, as it's illustrated by the next figure III.12, when the user choose a diagnostic option at the BDiagGUI, a diagRequest will be sent to service process through D-Bus Daemon.

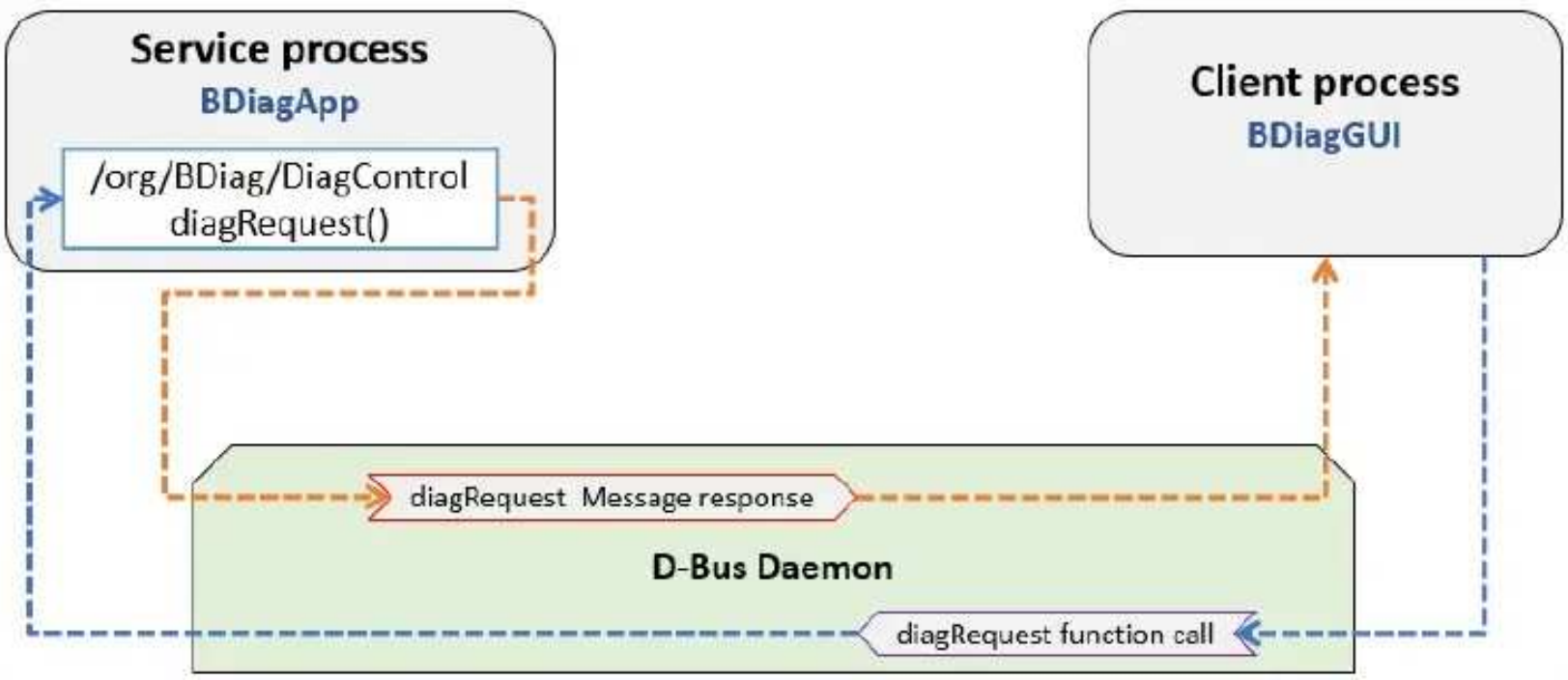


Figure III.12 – DBus

So, this function call will executed **diagRequest()** method at the **/org/BDiag/DiagControl** Object Path, in the BDiagApp service. this last, will invoke other function like :

— **cansend()** : to send diagnostic frames from the phyBOARD-Mira to the telematic ECU.
— **frameToMessage()** : the telematic ECU was configured to reply directly at every comes CAN frame, so this function will converted each one of this frames into a comprehensive message that will be transmitted and shown in the BDiagGUI interface.

Finally, when **diagRequest()** method was completely executed a response message will be send through the D-Bus Daemon from the BDiagApp to the BDiagGUI.

The entire process sounds like to be complex and difficult. But it can be spotted using some particular software designed for D-Bus services like **d-feet** figure III.13.

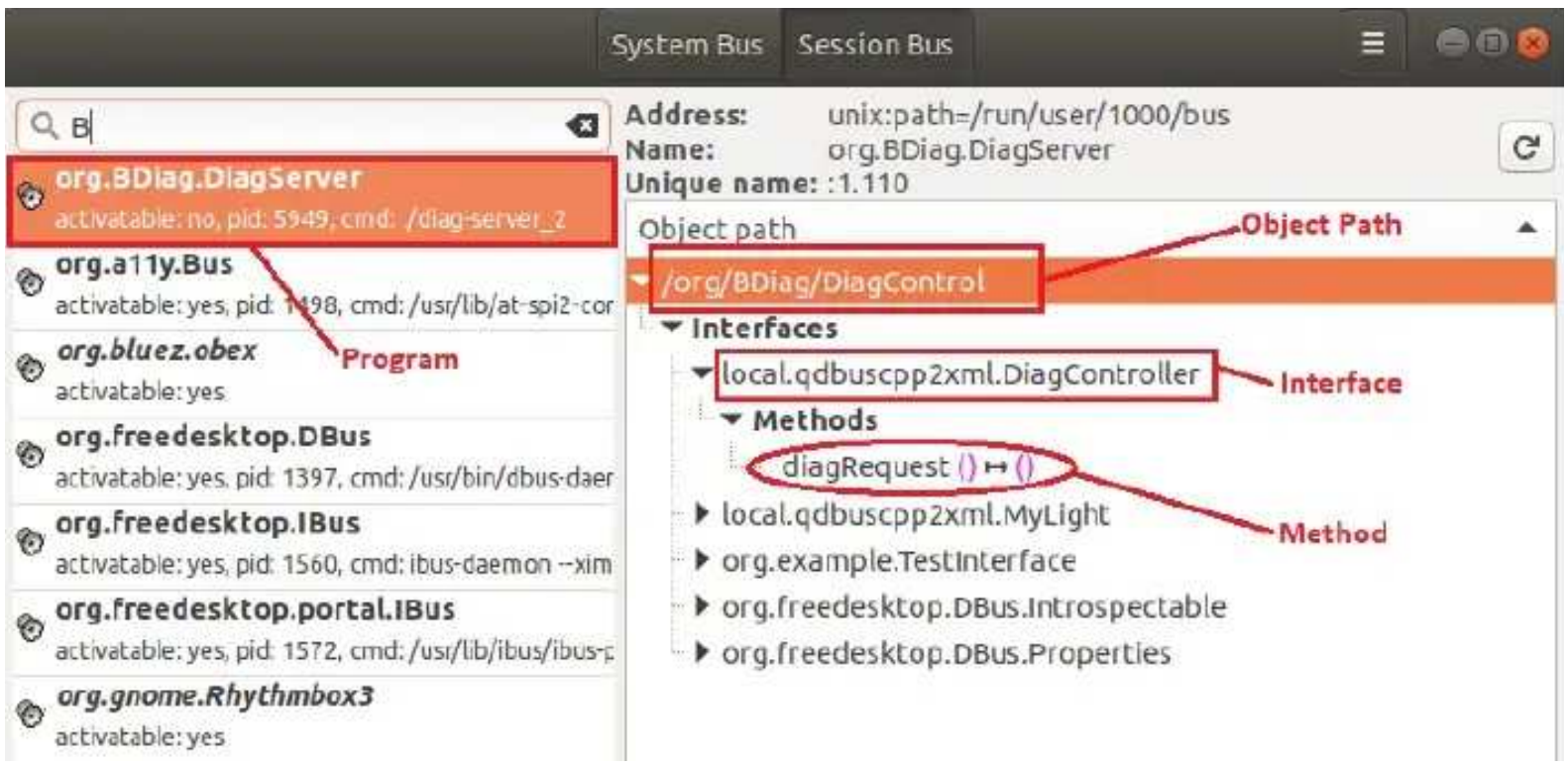


Figure III.13 – d-feet view

As shown in the d-feet view, our BDiagApp service **"org.BDiag.DiagServer"** has **"/org/BDiag/DiagControl"** as an Object Path. This object path contain our proper interface named **"local.qdbuscpp2xml.DiagController"** that was generated using Qt D-Bus XML compiler *qdbuscpp2xml* based on our **DiagController** Class.

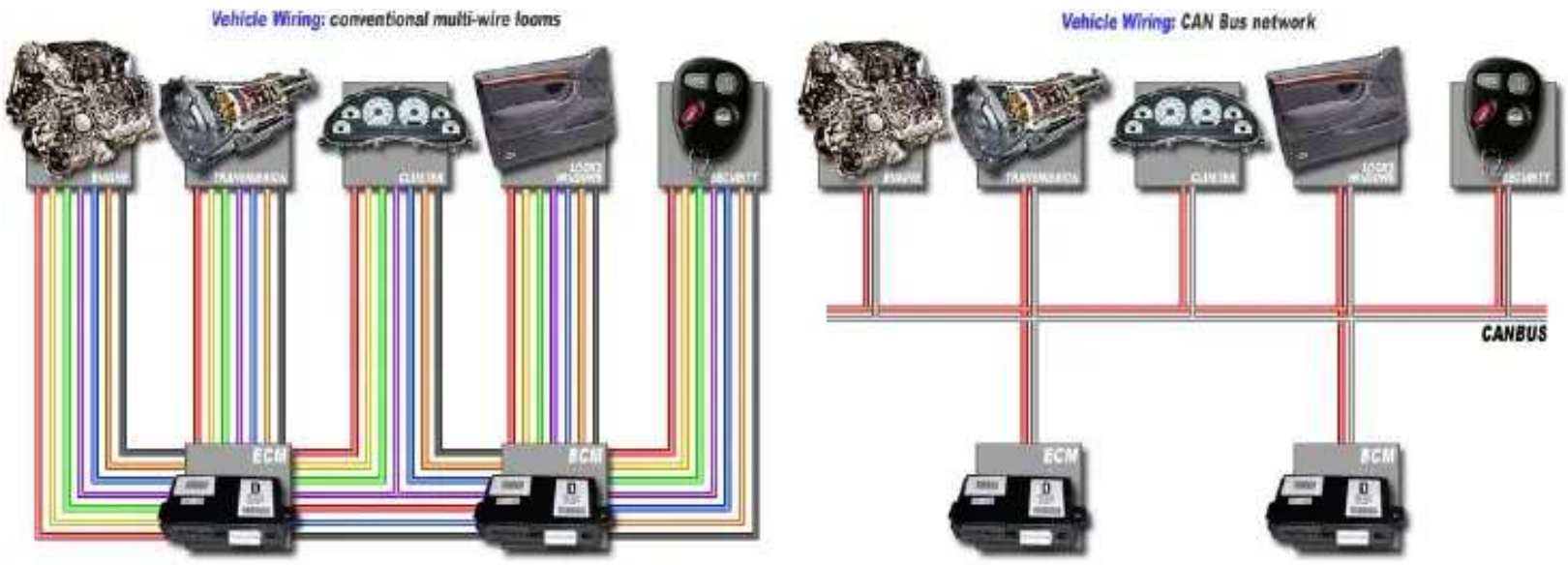
The output of this compiler was an XML file, listing III.2, that describe the different signals and methods of this interface. This file must be placed in booth programs BDiagGUI and BDiagApp, so they can communicate which each other using/through the D-Bus Daemon.

Listing III.2 – DiagController interface

```
"<node>\"
  "  <interface name='local.qdbuscpp2xml.DiagController'>\n"
  "    <method name='diagRequest'>\n"
  "    </method>\n"
  "    <signal name='diagFramChanged'>\n"
  "    </signal>\n"
"  </interface>\n"
"</node>\n";
```

3.2 Controller Area Network for ECU communication

The CAN Bus is an automotive bus elaborated by Robert Bosch in the early 80's years. CAN is the acronym of Controller Area Network. i'is a serial bus protocol used to connect individual sensors and systems as an alternative to traditional multi-wire looms
As the figure shows III.14a conventional multi-wire looms can be a real problem, especially when it comes to add another ECU to the system. this will launch a need of many wires connection to connect the new ECU with the other car components. This is maybe the most strong reason that CAN has quickly obtained acceptance into the automotive industries.



(a) conventional multi-wire **(b)** CAN Bus network

Figure III.14 – CAN Bus

As an example of the use of the CAN protocol in this vehicle, the BMW 850 coupe 1986 which considered as the first CAN Bus vehicle to enter the market. It reduced it's wiring by 2Km and that's lead to a notable reduction of the overall weight by at least 50Kg and using only half the connectors. So for the first time, each of the vehicle systems and sensors were capable to communicate at very high speeds (25kbps - 1Mbps) on a single or dual-wire connection line as opposed to the previous multi-wire looms.

3.3 CAN Bus on the phyBOARD-Mira

The phyBOARD-Mira-i.MX 6 offers a CAN interface figure, which is supported by drivers using the Linux "SocketCAN" framework standard. Using this framework, CAN interfaces can be programmed with the BSD socket API.

The SocketCAN framework for Linux extends the BSD socket API concept towards CAN bus. Because of that, using this framework, the CAN interfaces can be programmed with the BSD socket API and behaves like an ordinary Linux network device, with some additional features special to CAN.

Here, we will not talk about CAN itself as an electrical component (this will need an entire chapter), but we will focus on the Linux CAN driver to get successful communication between the phyBOARD-Mira and the telematic ECU.

First, we verified that can0 interface is up and ready for transmission, by executing the : target$ ***ip -d -s link show can0***

```
2: can0: <NOARP,ECHO> mtu 16 qdisc pfifo_fast state DOWN mode DEFAULT group default
qlen 10
    link/can  promiscuity 0
    can state STOPPED (berr-counter tx 0 rx 0) restart-ms 0
          bitrate 10000 sample-point 0.866
          tq 6666 prop-seg 6 phase-seg1 6 phase-seg2 2 sjw 1
          flexcan: tseg1 4..16 tseg2 2..8 sjw 1..4 brp 1..256 brp-inc 1
          clock 30000000
          re-started bus-errors arbit-lost error-warn error-pass bus-off
          0          0          0          0          0          0
    RX: bytes  packets  errors  dropped overrun mcast
    0          0        0       0       0       0
    TX: bytes  packets  errors  dropped carrier collsns
    0          0        0       0       0       0
```

Figure III.15 – CAN wave

The output contains a standard set of parameters like the *bitrate, clock* and *state.*

III.3 The engine of BDiag solution : "BDiagApp"

Next, we connected the CAN Bus outputs with an Oscilloscope. After sent a CAN frame message by executing cansend() program on the Mira board, A similar wave output (next figure III.16) should appear in the Oscilloscope display. This screen shots shows the outputs of CAN_H and CAN_L of the *can0* interface of the Mira board.



Figure III.16 – Mira CAN Bus output

At this point, the CAN Bus interface work correctly as its expected for the phyBOARD-Mira. But, its still need to ensure the reception and the transmission of CAN frames by the "telematic ECU". For that we used *kvaser leaf light v2* an external tool to sniffer to CAN Bus frames.

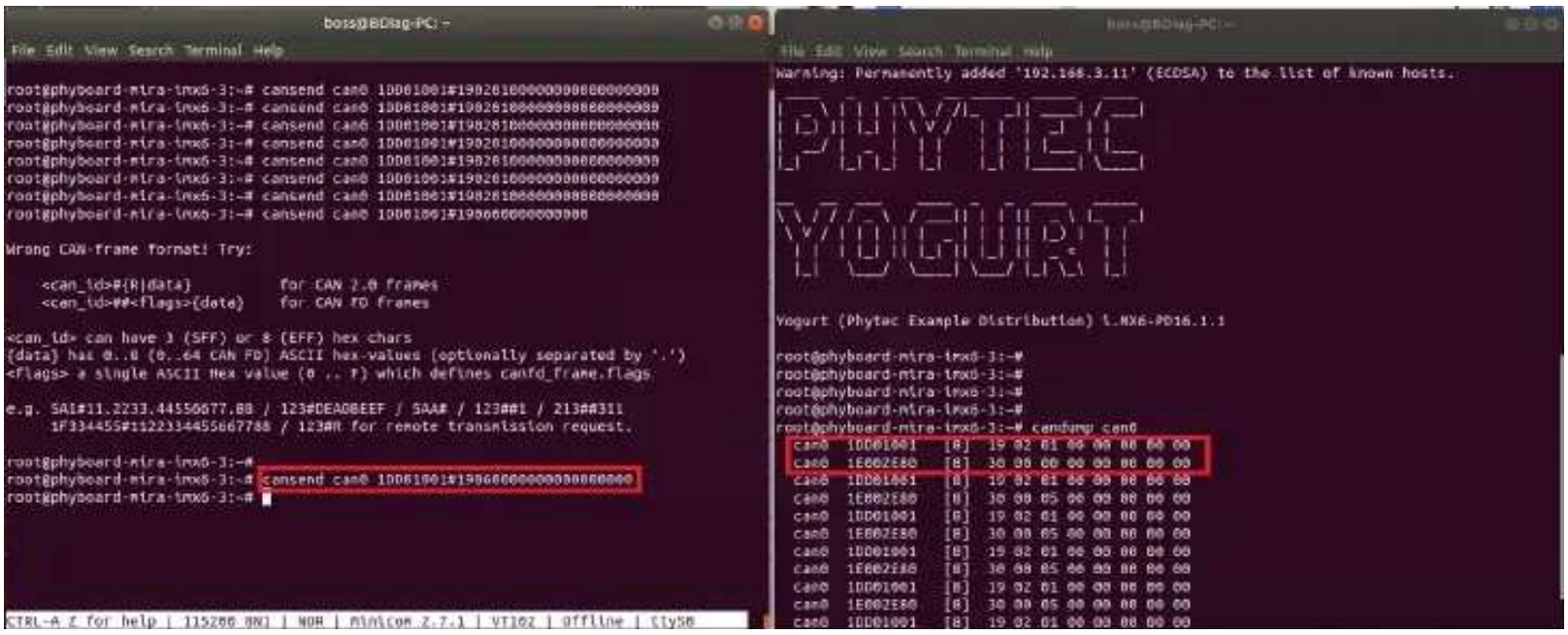


Figure III.17 – Mira and telematic ECU frames exchange

BDiag : A portable solution

In previous sections we talked about each part of the BDiag solution separately. Now its time to put those part together. For that, we backed again to the Yocto build system.
we created our proper layer **"meta-actia"** then we added our projects to this folder.
we modified the correspond "*.bb" files for each program, so in the build phase Yocto will take on consideration the use of some special library like the dbus lib for the BDiagApp.
After many tries of fixing the needed dependencies for each program, we ended by a unique image for the phyBOARD-Mira that contains booth the graphical and the diagnostic process of the BDiag Solution.
Absolutely true, this will be a great way to add the BDiag solution to existing products based on Linux OS, using the Yocto project.

In addition, Nowadays, the automotive market, presented many solutions that was based on Android operating system. For that, we added a new build Kit in Qt creator that allowed us to build the code project to a new device architecture Intel Atom x86_64 that works with Android system.

The next figure shows a variety of devices withe different architecture that running the BDiag solution, that's what we called **"A portable solution"**.



Figure III.18 – BDiag on Multiples platform

Conclusion

In this last chapter, we passed through the different parts of the implementation of the BDiag solution. In first phase we talked about the implementation of a graphical user interface "BDiagGUI", this includes the setup of Qt creator environment and cross-compiler for the phyBOARD-Mira. we finished this part by exposing some windows from the BDiagGUI application. In a second part, we spoken about "BDiagApp" the diagnostic process of the BDiag solution. This is based on the inter-process communication with D-Bus and ECU communication with the phyBOARD-Mira using CAN protocol.

General Conclusion

At the end of this report, we draw up a complete assessment of our work, carried out within ACTIA Engineering Services. The project entitled "*The embedded diagnostic and support solution in vehicles (**BDiag**)*" launched within the Yocto Project constitutes a step forwards towards end-to-end diagnostic solution in automotive industry.

Such an application can be very useful for many leaders in vehicles market. Its goal comes from pure strategic thinking and will, aims to provide useful service and great need for many companies and individuals essentially by losing old method of doing diagnostics with external tools and embedding the solution of the diagnostic solution in the system of the vehicles itself, and by then detecting anomalies and identifying error sources more efficiently.

Also, to emphasize the company's and our desire to create new solutions by embarking new technologies and bring new projects to the surface. The development was carried out according to an iterative and incremental methodology meaning, each iteration was approached by the drafting of a part of the functional and technical specifications. Then, the elaboration of the solution design and finally development and deployment of the system.

Our report was fenced by the exhibition of some screenshots of the work done and this document has detailed the steps taken to succeed in the work request.
We began with an exhaustive research in the fields of vehicle diagnostic. This research helped us to imagine, establish and find an efficient architecture featuring the system solution basically inspired from the used technologies, the statistical model approaches and previous works and prototypes.

In terms of work, the first step was the creation of a specific embedded Linux image for the phyBOARD-Mira board, that fulfilled our requirement and needs for the BDiag project. Next, we prepared a great development environment for the development of the graphical user interface. That's was done via the installation of Qt creator IDE and the creation of a new build kit based on the cross-toolchain that we generated using Yocto build system.

After that, we started the development of the BDiagGUI user interface. This first application was developed using "QML" language. This decision was justified by many raisins, and maybe, the portability of this language that can be deployed in every type of machine regardless of the OS, would be the most powerful reason for this choice.

Subsequently, we created a Git project and we linked it with three main repositories that contained all the work in progress. later, we took another important step in our project by getting a successful inter process communication between the interface application and the diagnostic process. This communication allowed us to carry on a call function and response message, between the BDiagGUI and the BDiagApp, despite of the different language that booth application based on.

Eventually, we tested the CAN bus serial interface on the phyBOARD-Mira and then we communicated the board with the telematic ECU. After that we backed again to Yocto build system, where we created our proper meta-layer and we added booth process to the generated image.

Finally, at the end of this graduation project, maybe we don't create a complete vehicle diagnostic solution. But for sure, we proved a new diagnostic architecture that takes advantages of new vehicle features like the telematic one. In addition we created a first prototype for this new diagnostic solution in a modular way, to make it reusable in future purpose. Besides, we demonstrated, that our solution can be deployed in a variety range of devices regardless of their processor architecture or their operating system.

References

[1] McKinsey and Company. *"Automotive revolution perspective towards 2030" (2016).* https://link.springer.com/content/pdf/10.1365/s40112-016-1117-8.pdf [In ligne ; last visit 19-August-2019].

[2] ACTIA Company. *"Multi-Diag Datasheet EN".* http://multidiag.com/images/PDF/Datasheet_MD360_EN.pdf [In ligne ; last visit 19-August-2019]. iv, 9, 10

[3] Hue Jie, Liu Changlin, Yan Fuwn, Zhang Zeyang. *"The Developement of Vehicle Diagnostic System Based on Android Platform".* 2015 International Conference on Connected Vehicles and Expo (ICCVE). iv, 10

[4] Tian Liyuan, Stevan Stevie, Zhang lindong *"Remote Diagnosis System of Vehicle Based on Telematics".* 2010 Interational Conference on Computer, Mechatronics, Control and Electronic Engineering (CMCE). 11

[5] Marko Dragojevie, Liu Changlin, Goran Stupar, Dusan Zivkov. *"Utilizing IoT Technologies for Remote Diagnostics of Next Generation Vehicles".* 2016 4th IEEE International Conference on Mobile Cloud Computing, Services, and Engineering. 11

[6] Jian XIONG, Hong GU. *"A Remote Engine Health Management System based on Mobile cloud computing".* 2018 IEEE 8th International Conference on Consumer Electronics - Berlin (ICCE-Berlin). 11

[7] VOLVO TRUCKS. https://www.volvotrucks.us/our-difference/uptime-and-connectivity/service-management-platform-asist/ [In ligne; last visit 19-August-2019]. 11

[8] Scania Assisting. https://www.scania.com/global/en/home/products-and-services/workshop-services/service-solutions/scania-assistance.html [In ligne ; last visit 19-August-2019]. 11

[9] Derek Molloy. *Exploring Raspberry Pi Interfacing to the Real World with Embedded Linux (2016).* Page 55-58. 24

[10] GNU Toolchain official website : http://www.gnu.org/software/hurd/toolchain.html. [In ligne; last visit 19-August-2019]. 25

[11] Chris Simmonds. *Mastering Embedded linux Programming - Second Edition (2017).* Page 21-22. 26

[12] Cross Linux From Scratch website : https://trac.clfs.org/. [In ligne; last visit 19-August-2019]. 27

References

[13] GNU Arm Embedded Toolchain website : https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm. [In ligne; last visit 19-August-2019]. 27

[14] Yocto Project official website : https://www.yoctoproject.org/. [In ligne; last visit 19-August-2019]. 28

[15] PHYTEC. *phyCORE-i.MX 6 Hardware Manual.* Document No. : L-808e-2. Edition : August 2016. Page 15-16. 31

[16] PHYTEC. *phyBOARD-Mira i.MX 6 Application Guide.* Document No. : L-806e-1. Edition : August 2015. Page 17-18. 31

[17] PHYTEC Wiki;. About the Yocto BSP; phyBOARD-Mira i.MX6 Linux Quickstart PD15.2.0. https://wiki.phytec.com. [In ligne; last visit 19-August-2019]. 34

[18] PHYTEC Wiki; Working Behind a Network Proxy. https://wiki.yoctoproject.org/wikiWorking_Behind_a_Network_Proxy. [In ligne; last visit 19-August-2019]. 36

[19] Jon Masters and Richard Blum. *Professional Linux Programming (2007).* Page 46. 40

Annex A : BDiag Mind Mapping

The next *"Mind Mapping"* gives a global idea of the principal tasks (yellow) and sub-tasks (green) needed to achieve this project. for example, our solution need a specific Linux image, and this last can be done throw some sub-tasks like the add of qt5 layer.

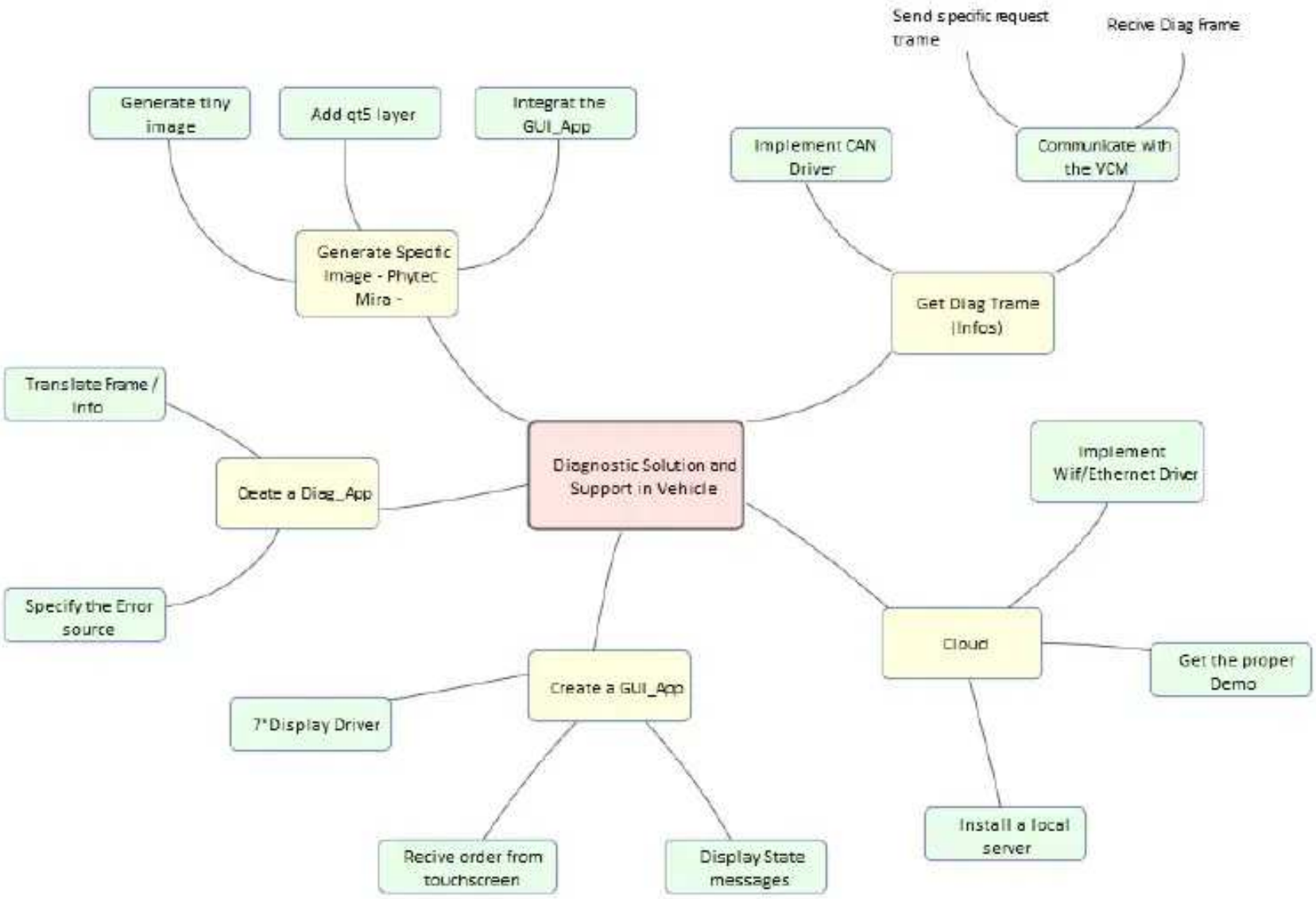


Figure A.1 – BDiag Mind Mapping

Annex B : phyCORE-i.MX6

Turn page

1 Introduction

The phyCORE-i.MX 6 belongs to PHYTEC's phyCORE System on Module family. The phyCORE SOMs represent the continuous development of PHYTEC System on Module technology. Like its mini-, micro- and nanoMODUL predecessors, the phyCORE boards integrate all core elements of a microcontroller system on a subminiature board and are designed in a manner that ensures their easy expansion and embedding in peripheral hardware developments.

As independent research indicates that approximately 70 % of all EMI (Electro Magnetic Interference) problems stem from insufficient supply voltage grounding of electronic components in high frequency environments, the phyCORE board design features an increased pin package. The increased pin package allows dedication of approximately 20 % of all connector pins on the phyCORE boards to Ground. This improves EMI and EMC characteristics and makes it easier to design complex applications meeting EMI and EMC guidelines using phyCORE boards even in high noise environments.

phyCORE boards achieve their small size through modern SMD technology and multi-layer design. In accordance with the complexity of the module, 0402-packaged SMD components and laser-drilled microvias are used on the boards; providing phyCORE users with access to this cutting edge miniaturization technology for integration into their own design.

The phyCORE-i.MX 6 is a subminiature (40 mm x 50 mm) insert-ready System on Module populated with the NXP® Semiconductor i.MX 6 microcontroller. Its universal design enables its insertion in a wide range of embedded applications. All controller signals and ports extend from the controller to high-density pitch (0.5 mm) connectors aligning two sides of the board, allowing it to be plugged like a "big chip" into a target application.

Precise specifications for the controller populating the board can be found in the applicable controller reference manual or datasheet. The descriptions in this manual are based on the NXP® Semiconductor i.MX 6. No description of compatible microcontroller derivative functions is included, as such functions are not relevant for the basic functioning of the phyCORE-i.MX 6.

1.1 Features of the phyCORE-i.MX 6

The phyCORE-i.MX 6 offers the following features:

- Insert-ready, sub-miniature (40 mm x 50 mm) System on Module (SOM) subassembly in low EMI design, achieved through advanced SMD technology
- Populated with the NXP® Semiconductor i.MX 6 microcontroller (BGA624 packaging)
- 1.0 GHz core clock frequency (optional 1.2 GHz)
- Boot from different memory devices (NAND Flash (standard))

- Controller signals and ports extend to two high-density (0.5 mm) Samtec connectors aligning two sides of the board, enabling the phyCORE-i.MX 6 to be plugged like a "big chip" into target application
- Single supply voltage of +3.3 V with on-board power management
- All controller required supplies are generated on board
- Improved interference safety achieved through multi-layer PCB technology and dedicated ground pins
- 512 MB (up to 2 GB[2]) DDR3 SDRAM
- 256 MB (up to 16 GB[2]) on-board NAND Flash
- Alternatively 2 GB (up to 32 GB[2]) on-board eMMC
- 16 MB[2] on-board serial Flash (bootable)
- 4 kB[2] I²C EEPROM
- Two serial interfaces (TTL). One with 4 lines allowing simple hardware handshake
- High-Speed USB OTG interface
- High-Speed USB HOST interface
- 10/100/1000 Mbit Ethernet interface. Either with Ethernet transceiver on the phyCORE-i.MX 6 allowing for direct connection to an existing Ethernet network, or without on-board transceiver and provision of the RMII signals at TTL-level at the phyCORE-Connector instead[3]
- I²C interface
- Two SPI interfaces
- PCIe interface
- I²S interface
- SPDIF interface
- PWM output
- CAN interface
- Two 4 channel LVDS (24 bit) LCD-interfaces
- Parallel LCD-interface with up to 24-bit
- HDMI interface
- Up to two parallel camera interfaces
- MIPI CSI camera interface
- Two SD/MMC card interfaces (1 x 8 bit, 1 x 4 bit)
- SATA interface
- JTAG interface
- One user programmable LED
- Several dedicated GPIOs[4]
- Power Management IC (PMIC)
- Available for different temperature grades (*section 17.1*)

---

[2]: The maximum memory size listed is as of the printing of this manual. Please contact PHYTEC for more information about additional, or new module configurations available.

[3]: Please refer to the order options described in the Preface, or contact PHYTEC for more information about additional module configurations.

[4]: Almost every controller port which connects directly to the phyCORE-Connector may be used as GPIO by using the i.MX 6's pin muxing options.

© PHYTEC Messtechnik GmbH 2016 L-808e_2

Annex C : phyBOARD-Mira i.MX6

Turn page

1 Introduction

1.1 Hardware Overview

The phyBOARD-Mira for phyCORE-i.MX 6 is a low-cost, feature-rich software development platform supporting the Freescale Semiconductor i.MX 6 microcontroller. Moreover, due to the numerous standard interfaces the phyBOARD-Mira i.MX 6 can serve as bedrock for your application. At the core of the phyBOARD-Mira is the PCM-058/phyCORE-i.MX 6 System On Module (SOM), containing the processor, DRAM, NAND Flash, power regulation, supervision, transceivers, and other core functions required to support the i.MX 6 processor. Surrounding the SOM is the PBA-CD-06/phyBOARD-Mira carrier board, adding power input, buttons, connectors, signal breakout, and Ethernet connectivity amongst other peripherals.

The PCM-058 System On Module connects to the phyBOARD-Mira carrier board by use of two high density connectors. The phyBOARD-Mira is also available with the phyCORE-i.MX 6 in a direct solder form factor (PCL-058), a connector-less, BGA style variant of the PCM-058/phyCORE-i.MX 6 SOM. The PCL-058 SOM is directly soldered down to the phyBOARD-Mira using PHYTEC's Direct Solder Connect technology. This solution offers an ultra-low cost Single Board Computer for the i.MX 6 processor, while maintaining most of the advantages of the SOM concept.

Adding the phyCORE-i.MX 6 SOM into your own design is as simple as ordering the connectored version (PCM-058) and making use of our phyCORE Carrier Board reference schematics.

1.1.1 Features of the phyBOARD-Mira i.MX 6

The phyBOARD-Mira i.MX 6 supports the following features :

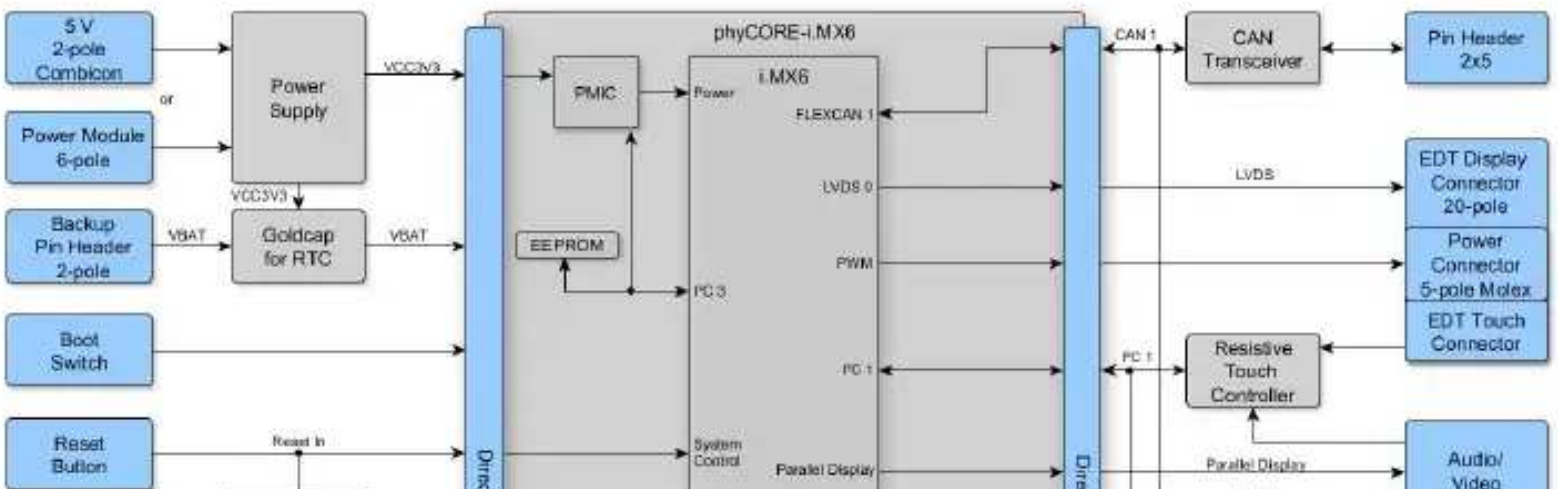
- Developed in accordance with PHYTEC's new SBCplus concept (*Preface*)
- PHYTEC's phyCORE-i.MX 6 SOM (optionally with Direct Solder Connect (DSC))
- Pico ITX standard dimensions (100 mm × 72 mm)
- Boot from MMC or NAND Flash
- Max. 1.2 GHz core clock frequency and up to four cores
- Two different power supply options (5 V via 3.5 mm combicon or 12 V – 24 V through external power module)
- One RJ45 jacks for 10/100/1000 Mbps Ethernet
- One USB host interface brought out to an upright USB Standard-A connector, or at the Mini PCI express connector[1]
- One USB OTG interface available at an USB Micro-AB connector, or at the expansion connector[1]

1 : **Caution!** There is no protective circuit for the USB interfaces brought out at the Mini PCI Express connector and the expansion connector.

- One Secure Digital / Multi Media Memory Card interface brought out to a Micro-SD connector at the back side
- CAN interface at 2×5 pin header 2.54 mm
- One HDMI interface brought out to a standard type A connector
- One LVDS interface brought out to a 20 pin FFC connector at the backside, and separate connector for backlight supply and control
- One touch interface at 1x4 pin header 2.54 mm

- One LVDS camera interfaces compatible to PHYTEC phyCAM-S+ camera standard with I^2C for camera control
- One PCI interface brought out to a Mini PCI Express connector, SIM-card signals are also available at the expansion connector
- RS-232 or RS-485 transceiver supporting UART3 incl. handshake signals with data rates of up to 1 Mbps (2×5 pin header 2.54 mm)
- Reset-Button
- One multicolor LED
- Audio/Video (A/V) connectors
- Expansion connector with different interfaces
- RTC
- Backup battery supply for RTC with external 2-pole pin-header or with Gold cap (lasts approx. 11 ½ days)

1.1.2 Block Diagram



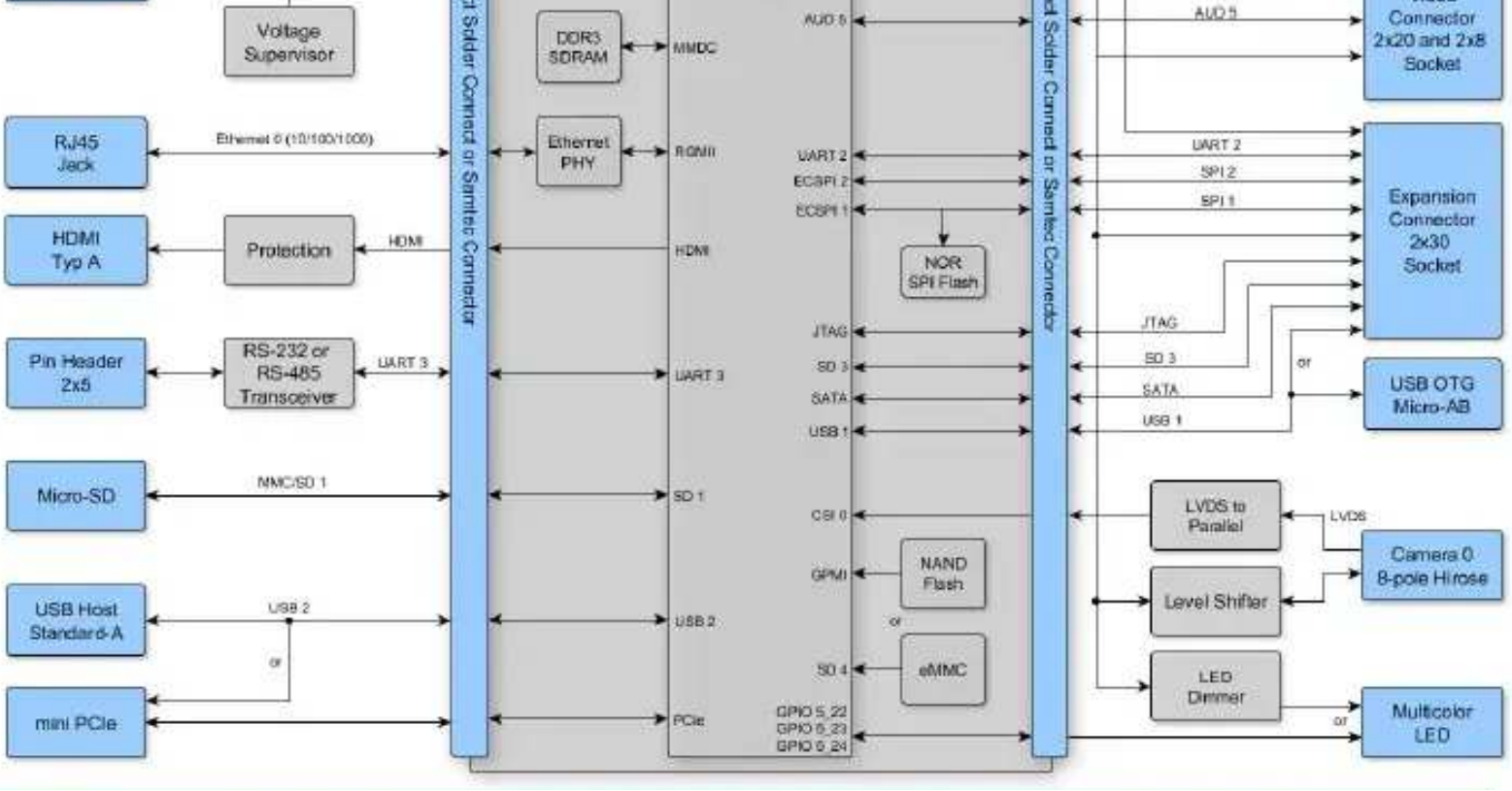


Figure 1: Block Diagram of the phyBOARD-Mira i.MX 6

Annex D : Git shell commands

The next git shell commands, listed the differnt operation that we can do on a git repository with a detailed description for each one.

OPERATION	DESCRIPTION	OPERATION	DESCRIPTION
`git clone`	Clone from the remote repository.	`git rm`	Delete a file or directory from the staging area.
`git init`	Create a wholly new repository.	`git mv`	Move or rename a file or folder in the staging area.
`git pull`	Merge changes from a	`git log`	Display a log of commits

`git pull`	Merge changes from a master repository.	`git log`	Display a log of commits. The project history.
`git fetch`	Find what has changed in a master repository without merging.	`git tag`	Give a commit a name (e.g., version 2).
`git status`	Show the project's status.	`git merge [name]`	Merge the branch.
`git add`	Add a new file or edit an existing file.	`git show`	Get details about the current or other commit.
`git diff`	Show the differences that are to be committed.	`git branch [name]`	Create a new branch. (Use `-d` to delete.)
`git commit`	Commit to the repository.	`git checkout [name]`	Switch to a different branch.
`git push`	Push changes from the local repository to a remote repository.		

Figure D.1 – Git shell commands